

CSC-3TC33

Algorithmique et fondements de l'informatique

Graphes - partie a

Bertrand Meyer

Automne 2024

Éléments de théorie des graphes

Graphes orientés

Définition

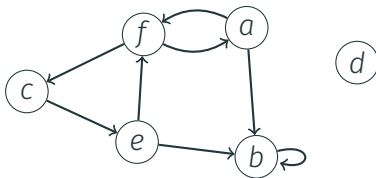
Un **graphe orienté** est un couple (V, E) où V est un ensemble fini et E une partie de $V \times V$.

Les éléments de V s'appellent les **sommets**; les éléments de E s'appellent les **arcs**.

Exemple

$V = \{a, b, c, d, e, f\}$

$E = \{ab, af, bb, ce, eb, ef, fa, fc\}$.



Quiz de vocabulaire (graphes orientés)

Terme

1. prédécesseur de v
2. degré entrant $d^-(v)$
3. successeur de v
4. degré sortant $d^+(v)$
5. chemin
6. circuit

Définition

- A. Suite de sommets $x_0x_1 \cdots x_k$ t.q. pour tout $i \in \llbracket 0, k \rrbracket$, $x_i x_{i+1} \in E$ et $x_k x_0 \in E$.
- B. Sommet u t.q. $vu \in E$
- C. $\text{Card}(\{u \in V; vu \in E\})$
- D. Suite de sommets $x_0x_1 \cdots x_k$ t.q. pour tout $i \in \llbracket 0, k \rrbracket$, $x_i x_{i+1} \in E$
- E. $\text{Card}(\{u \in V; uv \in E\})$
- F. Sommet u t.q. $uv \in E$

Quiz de vocabulaire (graphes orientés)

Terme

Définition

1. **prédécesseur** de v

2. **degré entrant** $d^-(v)$

3. **successeur** de v

4. **degré sortant** $d^+(v)$

5. **chemin**

6. **circuit**

A. Suite de sommets $x_0x_1 \cdots x_k$ t.q. pour tout $i \in \llbracket 0, k \rrbracket$, $x_i x_{i+1} \in E$ et $x_k x_0 \in E$.

B. Sommet u t.q. $vu \in E$

C. $\text{Card}(\{u \in V; vu \in E\})$

D. Suite de sommets $x_0x_1 \cdots x_k$ t.q. pour tout $i \in \llbracket 0, k \rrbracket$, $x_i x_{i+1} \in E$

E. $\text{Card}(\{u \in V; uv \in E\})$

F. Sommet u t.q. $uv \in E$

Réponse : 1 $\leftrightarrow$ F., 2 $\leftrightarrow$ E., 3 $\leftrightarrow$ B., 4 $\leftrightarrow$ C., 5 $\leftrightarrow$ D., 6 $\leftrightarrow$ A..

Graphes non orientés

Définition

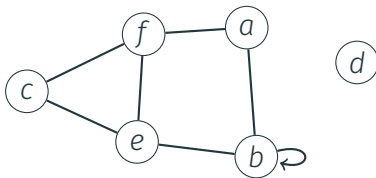
Un **graphe non orienté** est un graphe orienté (V, E) tel que pour tout arc $uv \in E$, vu est aussi un arc.

Les éléments de V s'appellent les **sommets**; les classes d'équivalence de E pour la relation $uv \equiv vu$ s'appellent les **arêtes**.

Exemple

$$V = \{a, b, c, d, e, f\}$$

$$E = \{ab, ba, af, fa, bb, be, eb, ce, ec, cf, fc, ef, fe\}.$$



Quiz de vocabulaire (graphes non orientés)

Définition

Terme

1. voisin

2. degré $d(v)$

3. chaîne

4. cycle

- A. Suite de sommets $x_0x_1 \cdots x_k$ t.q. pour tout $i \in \llbracket 0, k \rrbracket$, $x_i x_{i+1 \bmod k} \in E$ et $x_i \neq x_{i+2 \bmod k}$
- B. Sommets u et v t.q. $uv \in E$
- C. Suite de sommets $x_0x_1 \cdots x_k$ t.q. pour tout $i \in \llbracket 0, k \rrbracket$, $x_i x_{i+1} \in E$ et pour tout $i \in \llbracket 0, k-1 \rrbracket$, $x_i \neq x_{i+2}$
- D. $\text{Card}(\{u \in V; vu \in E\})$

Quiz de vocabulaire (graphes non orientés)

Définition

Terme

1. voisin

2. degré $d(v)$

3. chaîne

4. cycle

A. Suite de sommets $x_0x_1 \cdots x_k$ t.q. pour tout $i \in \llbracket 0, k \rrbracket$, $x_i x_{i+1 \bmod k} \in E$ et $x_i \neq x_{i+2 \bmod k}$

B. Sommets u et v t.q. $uv \in E$

C. Suite de sommets $x_0x_1 \cdots x_k$ t.q. pour tout $i \in \llbracket 0, k \rrbracket$, $x_i x_{i+1} \in E$ et pour tout $i \in \llbracket 0, k-1 \rrbracket$, $x_i \neq x_{i+2}$

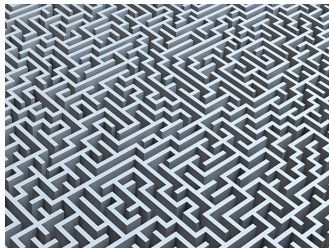
D. $\text{Card}(\{u \in V; vu \in E\})$

Réponse : 1 $\leftrightarrow$ B., 2 $\leftrightarrow$ D., 3 $\leftrightarrow$ C., 4 $\leftrightarrow$ A.,

Définition

Dans un graphe $G = (V, E)$, le sommet v est **accessible** depuis le sommet u s'il existe un chemin de u vers v .

On note $\rightsquigarrow$ la **relation d'accessibilité** dans $V \times V$.



Connexité dans un graphe non orienté

Lemme

Dans un graphe non orienté, $\rightsquigarrow$ est une relation d'équivalence.

Connexité dans un graphe non orienté

Lemme

Dans un graphe non orienté, $\rightsquigarrow$ est une relation d'équivalence.

Démonstration.

- Réflexivité : $v \rightsquigarrow v$ par un chemin de longueur nulle.

Connexité dans un graphe non orienté

Lemme

Dans un graphe non orienté, $\rightsquigarrow$ est une *relation d'équivalence*.

Démonstration.

- Réflexivité : $v \rightsquigarrow v$ par un chemin de longueur nulle.
- Symétrie : $u \rightsquigarrow v \Rightarrow v \rightsquigarrow u$ par symétrie des arcs.



Connexité dans un graphe non orienté

Lemme

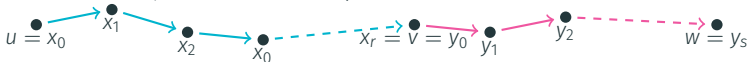
Dans un graphe non orienté, $\rightsquigarrow$ est une *relation d'équivalence*.

Démonstration.

- Réflexivité : $v \rightsquigarrow v$ par un chemin de longueur nulle.
- Symétrie : $u \rightsquigarrow v \Rightarrow v \rightsquigarrow u$ par symétrie des arcs.



- Transitivité : $u \rightsquigarrow v, v \rightsquigarrow w \Rightarrow u \rightsquigarrow w$ par concaténation des chemins.



□

Composantes connexes

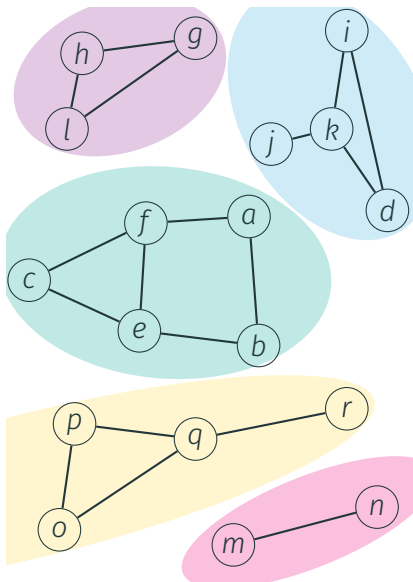
Définition

Les classes d'équivalences $V / \rightsquigarrow$ s'appellent les **composantes connexes** de G .

Un graphe est dit **connexe** s'il ne possède qu'une seule composante connexe.

Représentation

Type abstrait « partition »
implémenté avec **union-find**.



Vocabulaire : arbres (au sens de la théorie des graphes)

Définitions équivalentes

Soit $G = (V, E)$ un graphe. Notons

$n = |V|$ et $m = |E|$. On dit que G est un

arbre si [conditions équivalentes]

Vocabulaire : arbres (au sens de la théorie des graphes)

Définitions équivalentes

Soit $G = (V, E)$ un graphe. Notons

$n = |V|$ et $m = |E|$. On dit que G est un

arbre si [conditions équivalentes]

- G est acyclique et connexe

Vocabulaire : arbres (au sens de la théorie des graphes)

Définitions équivalentes

Soit $G = (V, E)$ un graphe. Notons

$n = |V|$ et $m = |E|$. On dit que G est un

arbre si [conditions équivalentes]

- G est acyclique et connexe
- G est connexe et $m = n - 1$

Vocabulaire : arbres (au sens de la théorie des graphes)

Définitions équivalentes

Soit $G = (V, E)$ un graphe. Notons

$n = |V|$ et $m = |E|$. On dit que G est un

arbre si [conditions équivalentes]

- G est acyclique et connexe
- G est connexe et $m = n - 1$
- G est acyclique et $m = n - 1$

Vocabulaire : arbres (au sens de la théorie des graphes)

Définitions équivalentes

Soit $G = (V, E)$ un graphe. Notons

$n = |V|$ et $m = |E|$. On dit que G est un

arbre si [conditions équivalentes]

- G est **acyclique et connexe**
- G est connexe et $m = n - 1$
- G est acyclique et $m = n - 1$
- G est connexe mais toute suppression d'arête déconnecte G

Vocabulaire : arbres (au sens de la théorie des graphes)

Définitions équivalentes

Soit $G = (V, E)$ un graphe. Notons

$n = |V|$ et $m = |E|$. On dit que G est un

arbre si [conditions équivalentes]

- G est acyclique et connexe
- G est connexe et $m = n - 1$
- G est acyclique et $m = n - 1$
- G est connexe mais toute suppression d'arête déconnecte G
- G est acyclique mais tout ajout d'arête crée un cycle

Vocabulaire : arbres (au sens de la théorie des graphes)

Définitions équivalentes

Soit $G = (V, E)$ un graphe. Notons

$n = |V|$ et $m = |E|$. On dit que G est un

arbre si [conditions équivalentes]

- G est **acyclique et connexe**
- G est connexe et $m = n - 1$
- G est acyclique et $m = n - 1$
- G est connexe mais toute suppression d'arête déconnecte G
- G est acyclique mais tout ajout d'arête crée un cycle
- entre deux sommets quelconques, il existe une chaîne élémentaire unique

Vocabulaire : arbres (au sens de la théorie des graphes)

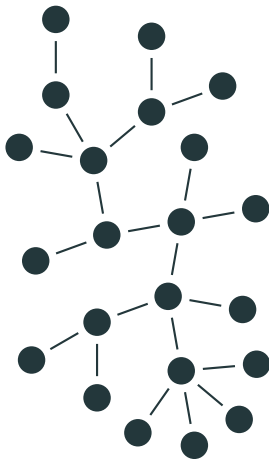
Définitions équivalentes

Soit $G = (V, E)$ un graphe. Notons

$n = |V|$ et $m = |E|$. On dit que G est un

arbre si [conditions équivalentes]

- G est **acyclique et connexe**
- G est connexe et $m = n - 1$
- G est acyclique et $m = n - 1$
- G est connexe mais toute suppression d'arête déconnecte G
- G est acyclique mais tout ajout d'arête crée un cycle
- entre deux sommets quelconques, il existe une chaîne élémentaire unique

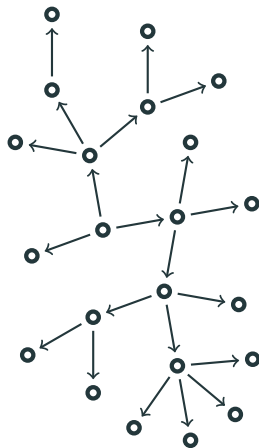


Vocabulaire : arborescence (au sens de la théorie des graphes)

Définition

On appelle **arborescence** de racine r tout graphe orienté $G = (V, E)$ tel que tout sommet $v \in V$ est accessible depuis r et tel que,

$$\deg^{-}(v) = \begin{cases} 1 & \text{si } v \in V \setminus \{r\} \\ 0 & \text{si } v = r \end{cases}$$



Vocabulaire : arborescence (au sens de la théorie des graphes)

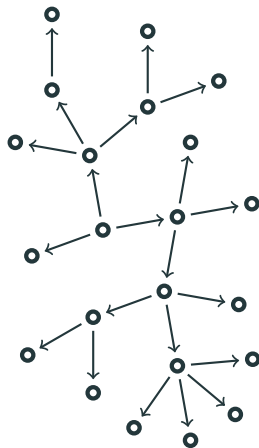
Définition

On appelle **arborescence** de racine r tout graphe orienté $G = (V, E)$ tel que tout sommet $v \in V$ est accessible depuis r et tel que,

$$\deg^{-}(v) = \begin{cases} 1 & \text{si } v \in V \setminus \{r\} \\ 0 & \text{si } v = r \end{cases}$$

Lemme

Dans une arborescence de racine r , pour tout $v \in V$, il existe un **unique chemin** de r vers v .



Vocabulaire : arborescence (au sens de la théorie des graphes)

Définition

On appelle **arborescence** de racine r tout graphe orienté $G = (V, E)$ tel que tout sommet $v \in V$ est accessible depuis r et tel que,

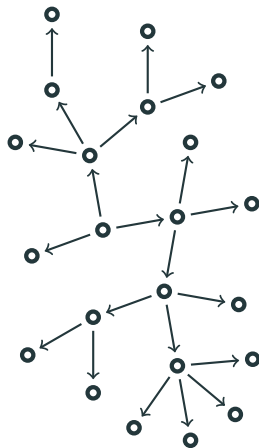
$$\deg^-(v) = \begin{cases} 1 & \text{si } v \in V \setminus \{r\} \\ 0 & \text{si } v = r \end{cases}$$

Lemme

Dans une arborescence de racine r , pour tout $v \in V$, il existe un **unique chemin** de r vers v .

Représentation

Dictionnaire sommet $\mapsto$ père.



Représentation des graphes

Trois modes de représentation

- Dictionnaire $\mathcal{D} : V \times V \rightarrow \mathbb{B}$
 $\mathcal{D}[u][v]$ vaut Vrai si (u, v) est un arc .
- Dictionnaire $\mathcal{T} : V \rightarrow 2^V$
 $\mathcal{T}[u] \subseteq V$ est l'ensemble des successeurs de v .
- Ensemble $\mathcal{E} \subseteq V^2$.
 $\mathcal{E}$ est l'ensemble des arcs.

Cas d'école

Les manuels scolaires supposent souvent $V = \llbracket 0, n - 1 \rrbracket$.

- Matrice $n \times n$ de booléens.
On parle de **matrice d'adjacence** .
- Tableau de listes chaînées d'entiers.
On parle de **listes d'adjacence** .
- Liste chaînée de couples d'entiers.

Représentation des graphes

Trois modes de représentation

- Dictionnaire $\mathcal{D} : V \times V \rightarrow \mathbb{B}$
 $\mathcal{D}[u][v]$ vaut Vrai si (u, v) est un arc .
- Dictionnaire $\mathcal{T} : V \rightarrow 2^V$
 $\mathcal{T}[u] \subseteq V$ est l'ensemble des successeurs de v .
- Ensemble $\mathcal{E} \subseteq V^2$.
 $\mathcal{E}$ est l'ensemble des arcs.

Remarque

Dans un réseau, un sommet ne connaît pas tout le graphe : seul le modèle « listes d'adjacence » est réaliste.

Cas d'école

Les manuels scolaires supposent souvent $V = \llbracket 0, n - 1 \rrbracket$.

- Matrice $n \times n$ de booléens.
On parle de **matrice d'adjacence** .
- Tableau de listes chaînées d'entiers.
On parle de **listes d'adjacence** .
- Liste chaînée de couples d'entiers.

Parcours de graphe

La notion de parcours

Définition

Parcourir un graphe signifie *visiter* tous les sommets d'un graphe accessibles depuis un premier sommet en se déplaçant de voisins en voisins et en *marquant* les sommets visités pour ne pas les voir plusieurs fois.

Remarque

En notant $\text{père}(v)$ le sommet u depuis lequel on a découvert le sommet v , on obtient une arborescence, dite **arborescence de parcours** qui coïncide avec l'ensemble des sommets accessibles.

Parcours en profondeur (version récursive)

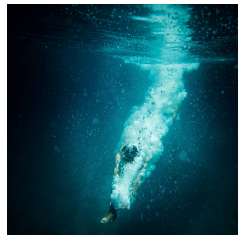
Le **parcours en profondeur** (ou DFS, comme *Depth First Search*) du graphe peut se décrire comme suit :

Algorithme 1 : Parcours en profondeur

Entrées : Graphe $G = (V, E)$, sommet s

Résultat : Visite des sommets accessibles
depuis s

- 1 Marquer s
 - 2 **pour** w voisin de s non encore marqué **faire**
 - 3 $\text{père}(w) \leftarrow s$
 - 4 DFS(w)
-



(le recours à une structure de pile est caché dans les appels récursifs)

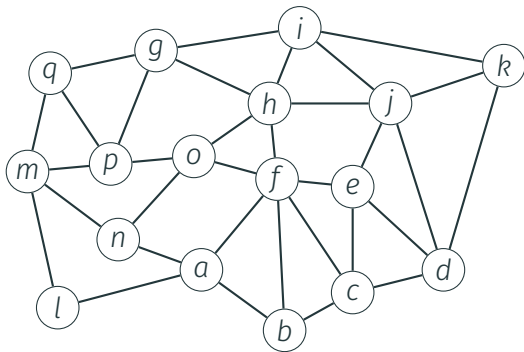
Implémentation du DFS en python

```
class Graph():
    # Les sommets sont numérotés de 0 à n-1
    # Les arcs sont stockés par listes de voisins
    def __init__(self, n):
        self.neighbors = [[] for _ in range(n)]
    def add_edge(self, u, v):
        self.neighbors[u].append(v)
        self.neighbors[v].append(u)
    def dfs(self, r, parent=None):
        if parent is None:
            parent = dict({r:-1})
        for u in self.neighbors[r]:
            if u not in parent:
                parent[u] = r
                self.dfs(u, parent)
        return parent
```

DFS : illustration

Exemple

Parcours en profondeur au départ du sommet n .

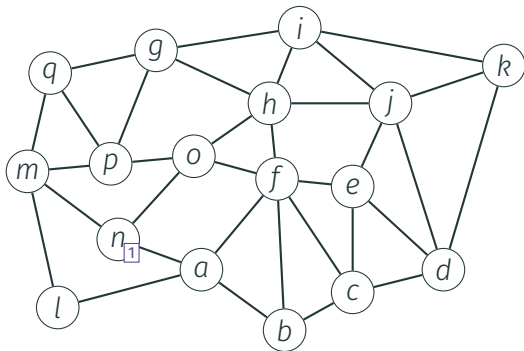


DFS : illustration

Exemple

Parcours en profondeur au départ du sommet n .

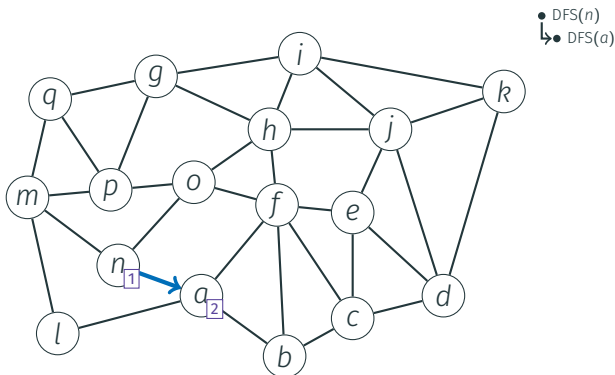
• DFS(n)



DFS : illustration

Exemple

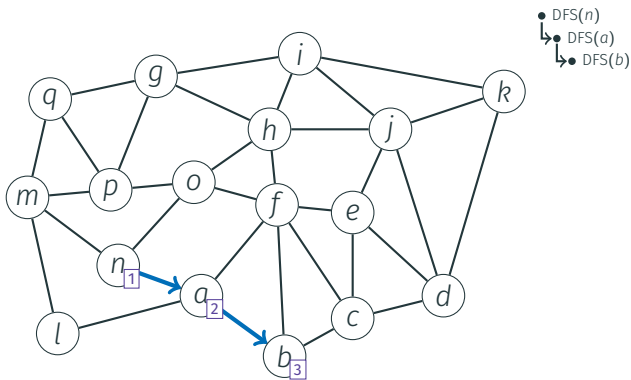
Parcours en profondeur au départ du sommet n .



DFS : illustration

Exemple

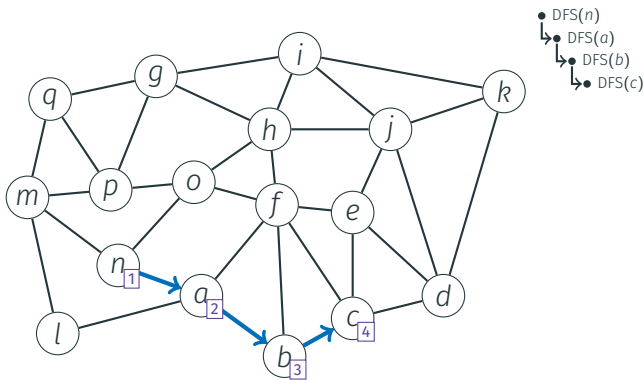
Parcours en profondeur au départ du sommet n .



DFS : illustration

Exemple

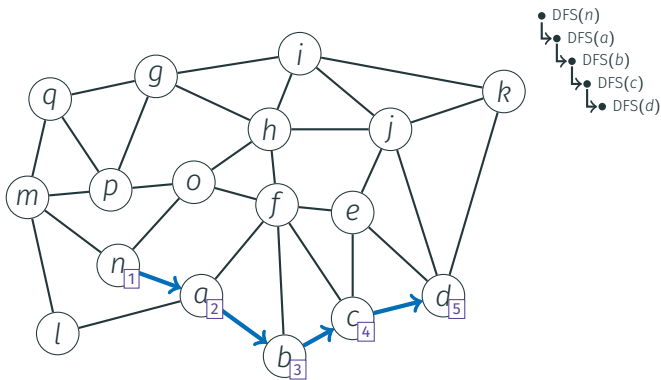
Parcours en profondeur au départ du sommet n .



DFS : illustration

Exemple

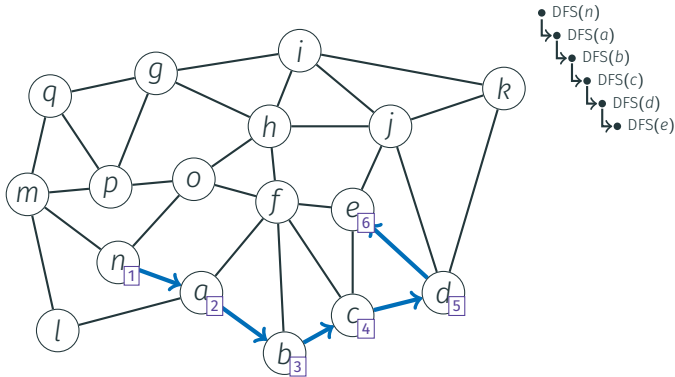
Parcours en profondeur au départ du sommet n .



DFS : illustration

Exemple

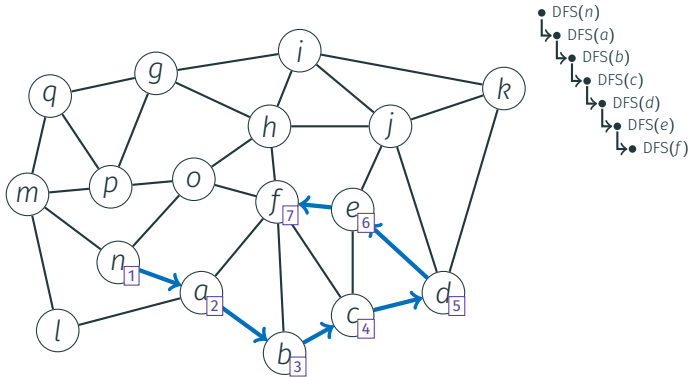
Parcours en profondeur au départ du sommet n .



DFS : illustration

Exemple

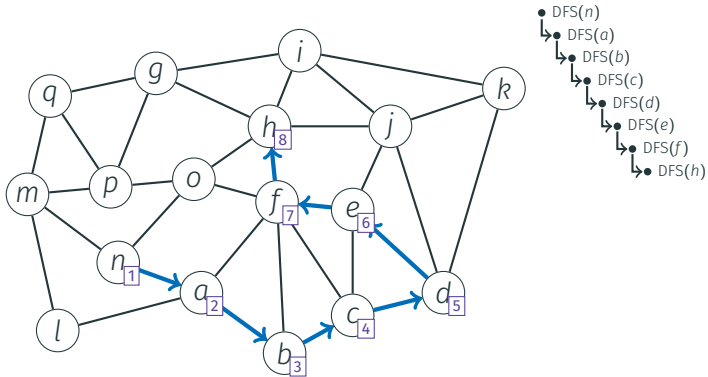
Parcours en profondeur au départ du sommet n .



DFS : illustration

Exemple

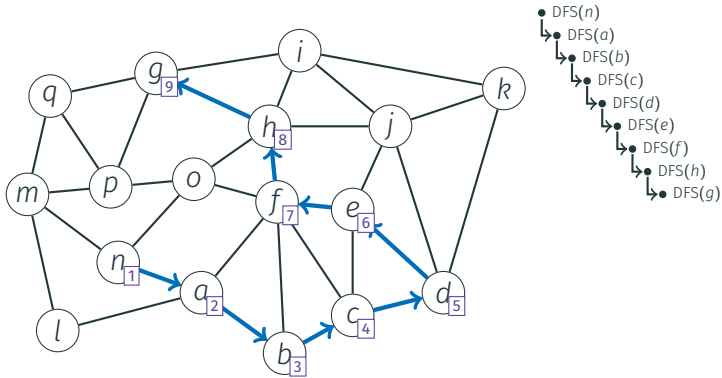
Parcours en profondeur au départ du sommet n .



DFS : illustration

Exemple

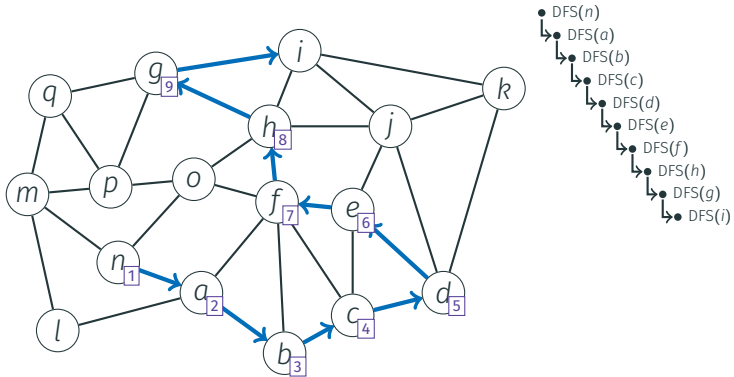
Parcours en profondeur au départ du sommet n .



DFS : illustration

Exemple

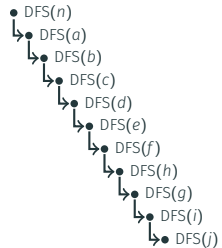
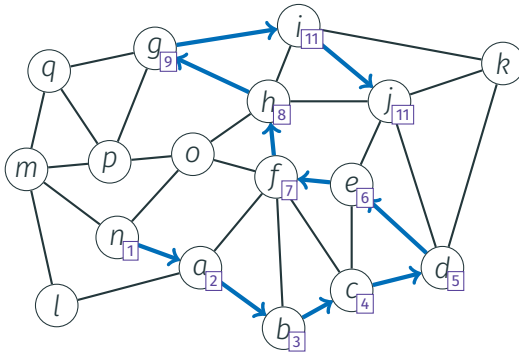
Parcours en profondeur au départ du sommet n .



DFS : illustration

Exemple

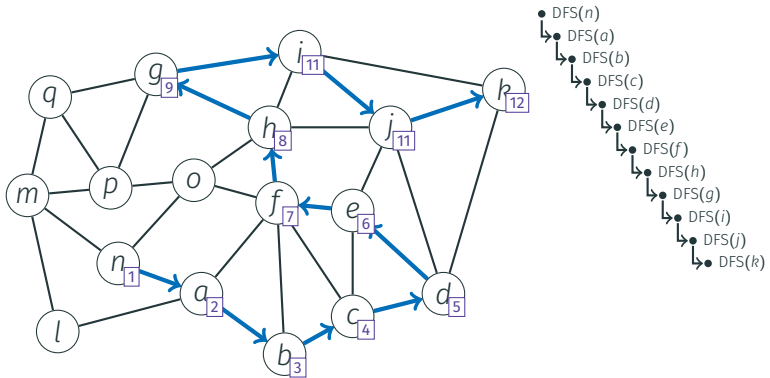
Parcours en profondeur au départ du sommet n .



DFS : illustration

Exemple

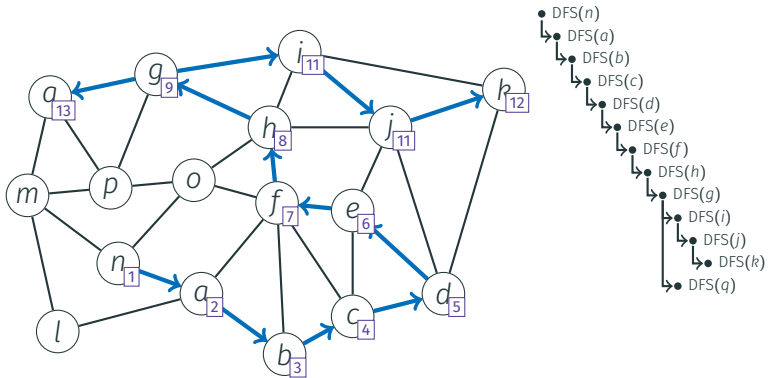
Parcours en profondeur au départ du sommet n .



DFS : illustration

Exemple

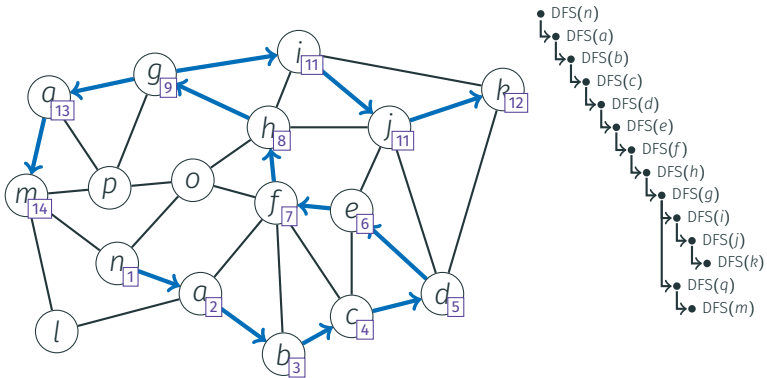
Parcours en profondeur au départ du sommet n .



DFS : illustration

Example

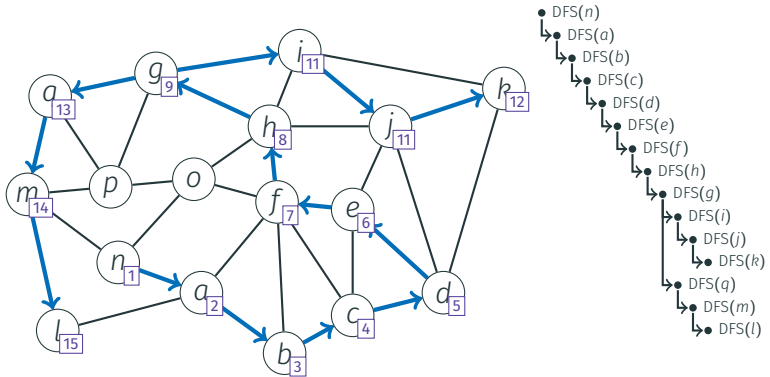
Parcours en profondeur au départ du sommet n .



DFS : illustration

Exemple

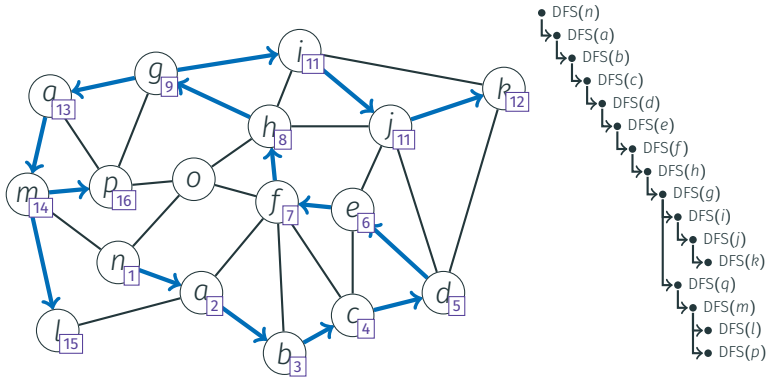
Parcours en profondeur au départ du sommet n .



DFS : illustration

Exemple

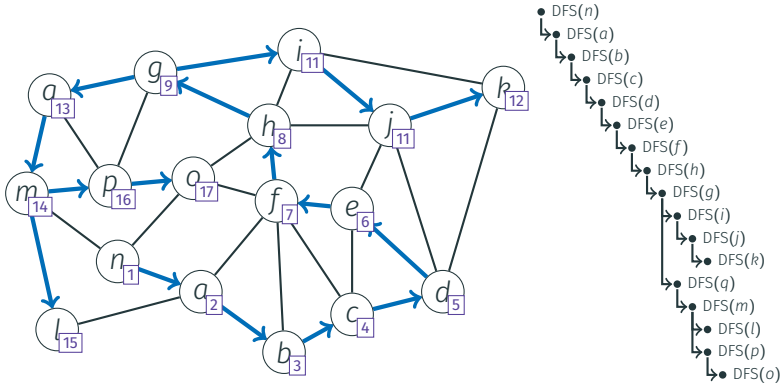
Parcours en profondeur au départ du sommet n .



DFS : illustration

Example

Parcours en profondeur au départ du sommet n .

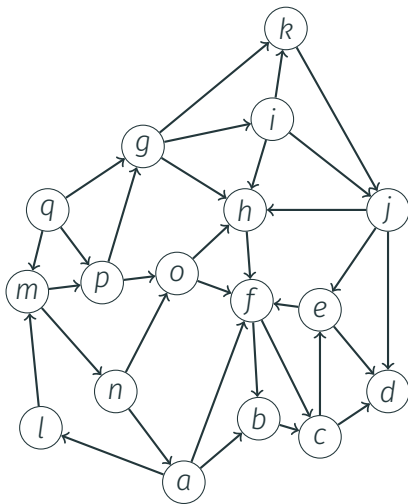


Classification de l'arborescence de parcours

Définition

Après un DFS, en notant (V, A) l'arborescence de parcours, on distingue les

- **arcs arborescents** : arcs de A
- **arcs avant** : uv où v est un descendant de u dans (V, A)
- **arcs arrière** : uv où u est un descendant de v dans (V, A)
- **arcs transverses** : les autres.

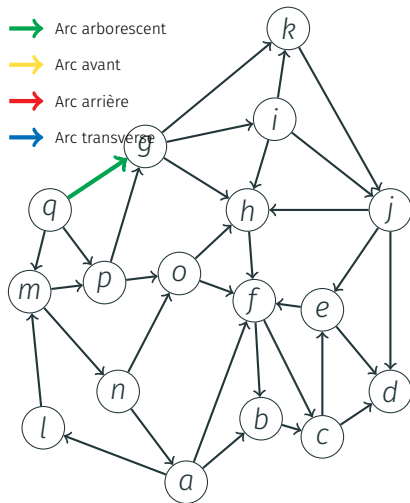


Classification de l'arborescence de parcours

Définition

Après un DFS, en notant (V, A) l'arborescence de parcours, on distingue les

- **arcs arborescents** : arcs de A
- **arcs avant** : uv où v est un descendant de u dans (V, A)
- **arcs arrière** : uv où u est un descendant de v dans (V, A)
- **arcs transverses** : les autres.

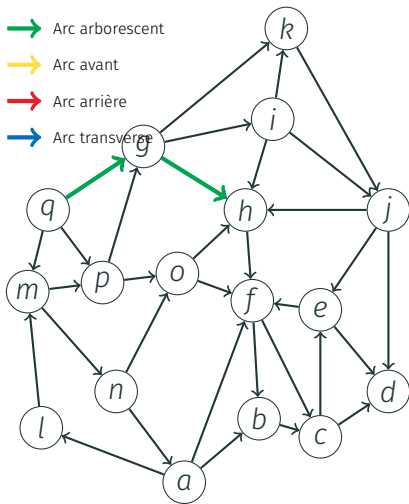


Classification de l'arborescence de parcours

Définition

Après un DFS, en notant (V, A) l'arborescence de parcours, on distingue les

- arcs arborescents : arcs de A
- arcs avant : uv où v est un descendant de u dans (V, A)
- arcs arrière : uv où u est un descendant de v dans (V, A)
- arcs transverses : les autres.

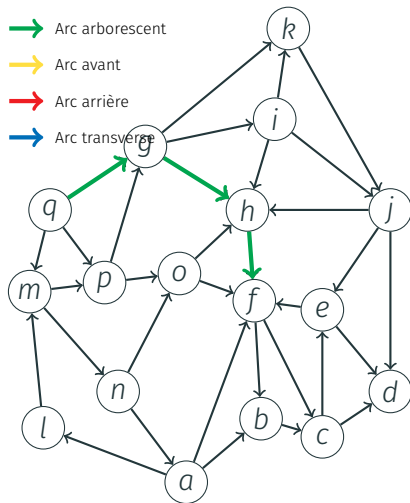


Classification de l'arborescence de parcours

Définition

Après un DFS, en notant (V, A) l'arborescence de parcours, on distingue les

- **arcs arborescents** : arcs de A
- **arcs avant** : uv où v est un descendant de u dans (V, A)
- **arcs arrière** : uv où u est un descendant de v dans (V, A)
- **arcs transverses** : les autres.

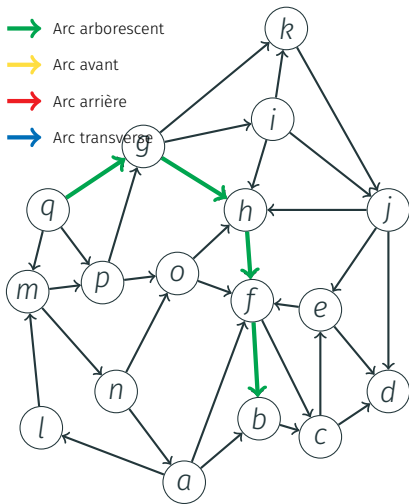


Classification de l'arborescence de parcours

Définition

Après un DFS, en notant (V, A) l'arborescence de parcours, on distingue les

- arcs arborescents : arcs de A
- arcs avant : uv où v est un descendant de u dans (V, A)
- arcs arrière : uv où u est un descendant de v dans (V, A)
- arcs transverses : les autres.

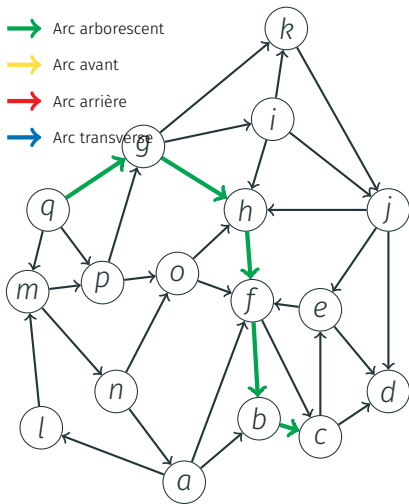


Classification de l'arborescence de parcours

Définition

Après un DFS, en notant (V, A) l'arborescence de parcours, on distingue les

- arcs arborescents : arcs de A
- arcs avant : uv où v est un descendant de u dans (V, A)
- arcs arrière : uv où u est un descendant de v dans (V, A)
- arcs transverses : les autres.

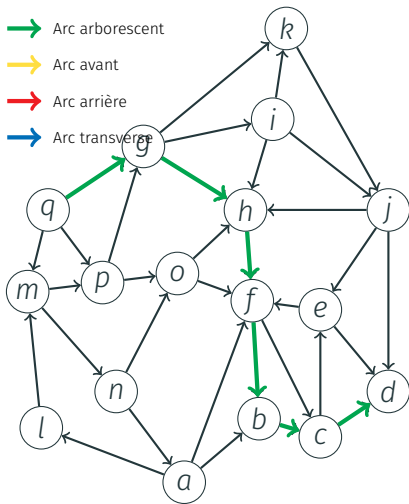


Classification de l'arborescence de parcours

Définition

Après un DFS, en notant (V, A) l'arborescence de parcours, on distingue les

- **arcs arborescents** : arcs de A
- **arcs avant** : uv où v est un descendant de u dans (V, A)
- **arcs arrière** : uv où u est un descendant de v dans (V, A)
- **arcs transverses** : les autres.

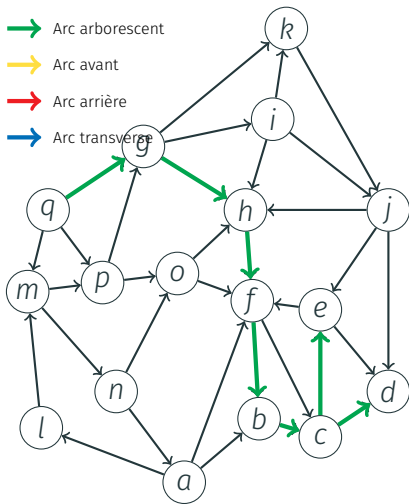


Classification de l'arborescence de parcours

Définition

Après un DFS, en notant (V, A) l'arborescence de parcours, on distingue les

- **arcs arborescents** : arcs de A
- **arcs avant** : uv où v est un descendant de u dans (V, A)
- **arcs arrière** : uv où u est un descendant de v dans (V, A)
- **arcs transverses** : les autres.

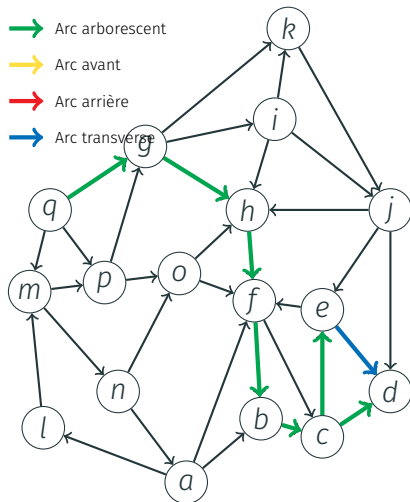


Classification de l'arborescence de parcours

Définition

Après un DFS, en notant (V, A) l'arborescence de parcours, on distingue les

- **arcs arborescents** : arcs de A
- **arcs avant** : uv où v est un descendant de u dans (V, A)
- **arcs arrière** : uv où u est un descendant de v dans (V, A)
- **arcs transverses** : les autres.

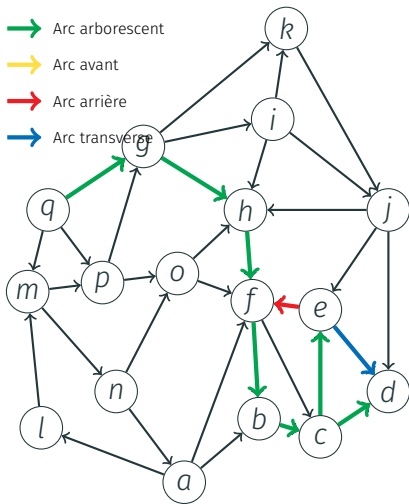


Classification de l'arborescence de parcours

Définition

Après un DFS, en notant (V, A) l'arborescence de parcours, on distingue les

- **arcs arborescents** : arcs de A
- **arcs avant** : uv où v est un descendant de u dans (V, A)
- **arcs arrière** : uv où u est un descendant de v dans (V, A)
- **arcs transverses** : les autres.

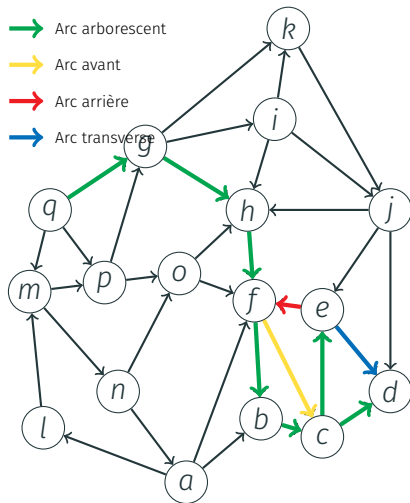


Classification de l'arborescence de parcours

Définition

Après un DFS, en notant (V, A) l'arborescence de parcours, on distingue les

- **arcs arborescents** : arcs de A
- **arcs avant** : uv où v est un descendant de u dans (V, A)
- **arcs arrière** : uv où u est un descendant de v dans (V, A)
- **arcs transverses** : les autres.

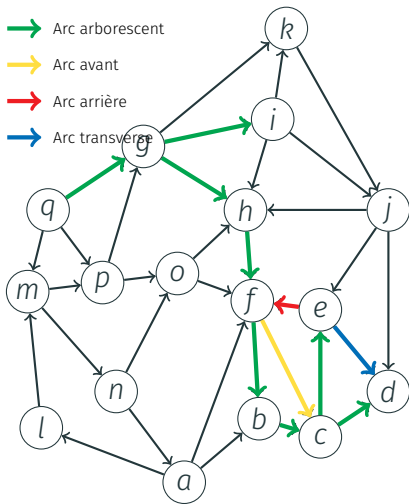


Classification de l'arborescence de parcours

Définition

Après un DFS, en notant (V, A) l'arborescence de parcours, on distingue les

- **arcs arborescents** : arcs de A
- **arcs avant** : uv où v est un descendant de u dans (V, A)
- **arcs arrière** : uv où u est un descendant de v dans (V, A)
- **arcs transverses** : les autres.

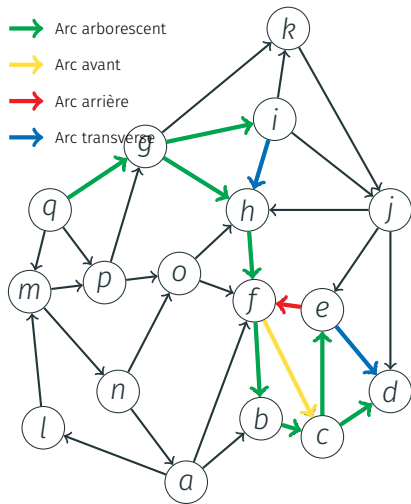


Classification de l'arborescence de parcours

Définition

Après un DFS, en notant (V, A) l'arborescence de parcours, on distingue les

- **arcs arborescents** : arcs de A
- **arcs avant** : uv où v est un descendant de u dans (V, A)
- **arcs arrière** : uv où u est un descendant de v dans (V, A)
- **arcs transverses** : les autres.

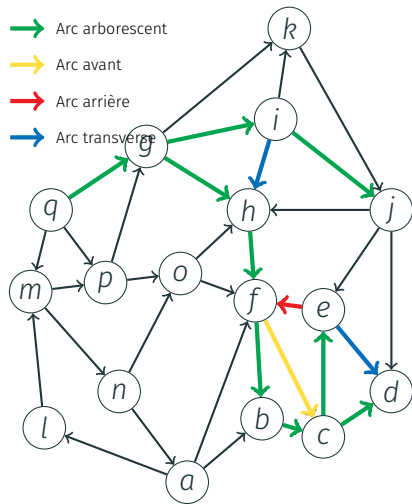


Classification de l'arborescence de parcours

Définition

Après un DFS, en notant (V, A) l'arborescence de parcours, on distingue les

- **arcs arborescents** : arcs de A
- **arcs avant** : uv où v est un descendant de u dans (V, A)
- **arcs arrière** : uv où u est un descendant de v dans (V, A)
- **arcs transverses** : les autres.

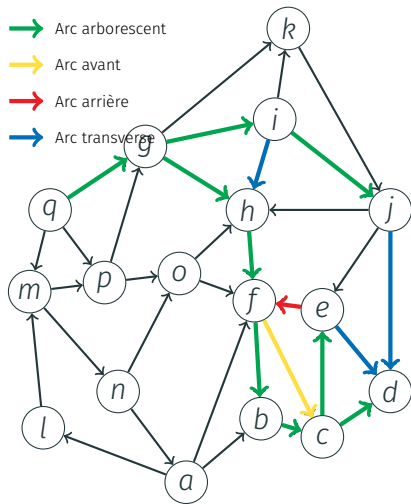


Classification de l'arborescence de parcours

Définition

Après un DFS, en notant (V, A) l'arborescence de parcours, on distingue les

- **arcs arborescents** : arcs de A
- **arcs avant** : uv où v est un descendant de u dans (V, A)
- **arcs arrière** : uv où u est un descendant de v dans (V, A)
- **arcs transverses** : les autres.

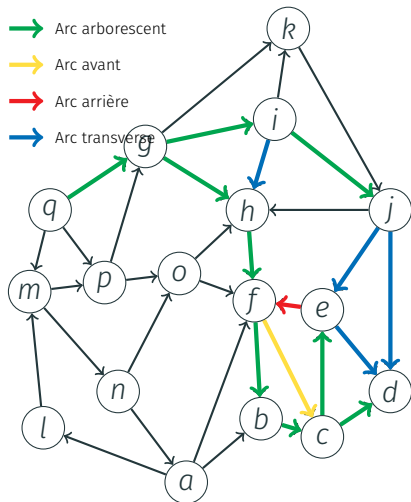


Classification de l'arborescence de parcours

Définition

Après un DFS, en notant (V, A) l'arborescence de parcours, on distingue les

- **arcs arborescents** : arcs de A
- **arcs avant** : uv où v est un descendant de u dans (V, A)
- **arcs arrière** : uv où u est un descendant de v dans (V, A)
- **arcs transverses** : les autres.

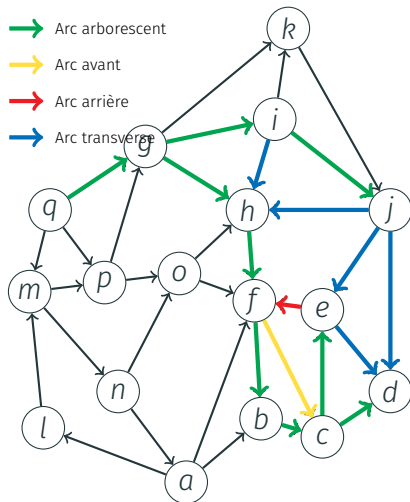


Classification de l'arborescence de parcours

Définition

Après un DFS, en notant (V, A) l'arborescence de parcours, on distingue les

- **arcs arborescents** : arcs de A
- **arcs avant** : uv où v est un descendant de u dans (V, A)
- **arcs arrière** : uv où u est un descendant de v dans (V, A)
- **arcs transverses** : les autres.

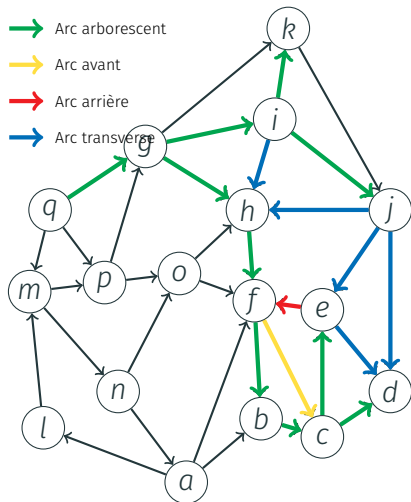


Classification de l'arborescence de parcours

Définition

Après un DFS, en notant (V, A) l'arborescence de parcours, on distingue les

- **arcs arborescents** : arcs de A
- **arcs avant** : uv où v est un descendant de u dans (V, A)
- **arcs arrière** : uv où u est un descendant de v dans (V, A)
- **arcs transverses** : les autres.

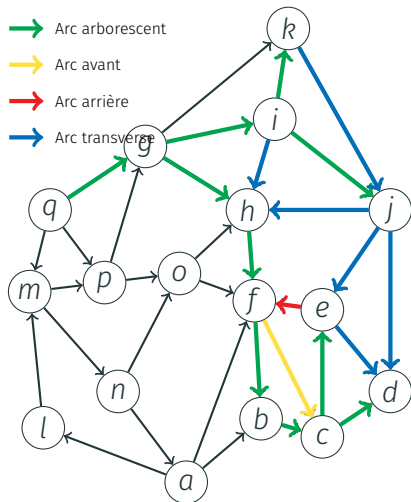


Classification de l'arborescence de parcours

Définition

Après un DFS, en notant (V, A) l'arborescence de parcours, on distingue les

- **arcs arborescents** : arcs de A
- **arcs avant** : uv où v est un descendant de u dans (V, A)
- **arcs arrière** : uv où u est un descendant de v dans (V, A)
- **arcs transverses** : les autres.

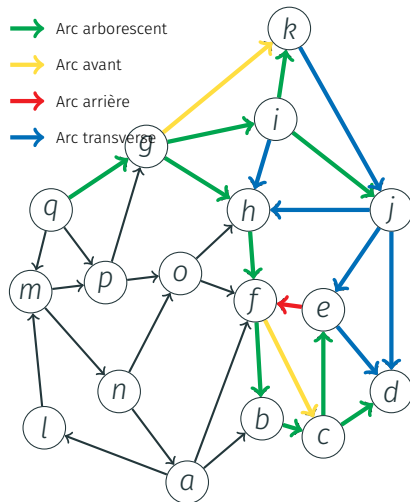


Classification de l'arborescence de parcours

Définition

Après un DFS, en notant (V, A) l'arborescence de parcours, on distingue les

- **arcs arborescents** : arcs de A
- **arcs avant** : uv où v est un descendant de u dans (V, A)
- **arcs arrière** : uv où u est un descendant de v dans (V, A)
- **arcs transverses** : les autres.

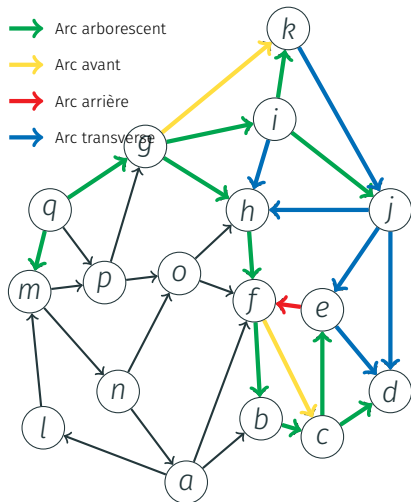


Classification de l'arborescence de parcours

Définition

Après un DFS, en notant (V, A) l'arborescence de parcours, on distingue les

- **arcs arborescents** : arcs de A
- **arcs avant** : uv où v est un descendant de u dans (V, A)
- **arcs arrière** : uv où u est un descendant de v dans (V, A)
- **arcs transverses** : les autres.

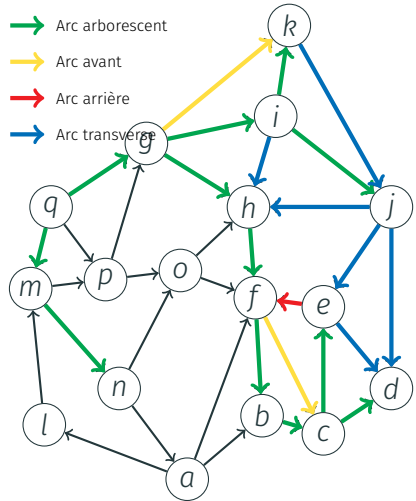


Classification de l'arborescence de parcours

Définition

Après un DFS, en notant (V, A) l'arborescence de parcours, on distingue les

- **arcs arborescents** : arcs de A
- **arcs avant** : uv où v est un descendant de u dans (V, A)
- **arcs arrière** : uv où u est un descendant de v dans (V, A)
- **arcs transverses** : les autres.

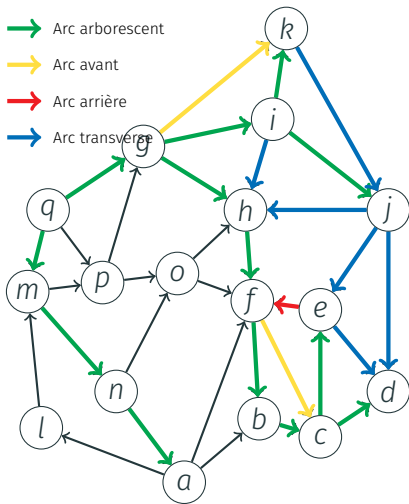


Classification de l'arborescence de parcours

Définition

Après un DFS, en notant (V, A) l'arborescence de parcours, on distingue les

- arcs arborescents : arcs de A
- arcs avant : uv où v est un descendant de u dans (V, A)
- arcs arrière : uv où u est un descendant de v dans (V, A)
- arcs transverses : les autres.

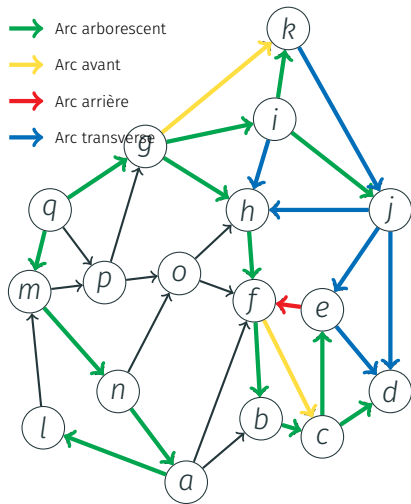


Classification de l'arborescence de parcours

Définition

Après un DFS, en notant (V, A) l'arborescence de parcours, on distingue les

- **arcs arborescents** : arcs de A
- **arcs avant** : uv où v est un descendant de u dans (V, A)
- **arcs arrière** : uv où u est un descendant de v dans (V, A)
- **arcs transverses** : les autres.

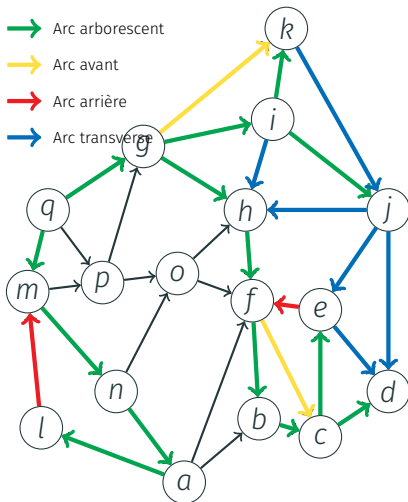


Classification de l'arborescence de parcours

Définition

Après un DFS, en notant (V, A) l'arborescence de parcours, on distingue les

- **arcs arborescents** : arcs de A
- **arcs avant** : uv où v est un descendant de u dans (V, A)
- **arcs arrière** : uv où u est un descendant de v dans (V, A)
- **arcs transverses** : les autres.

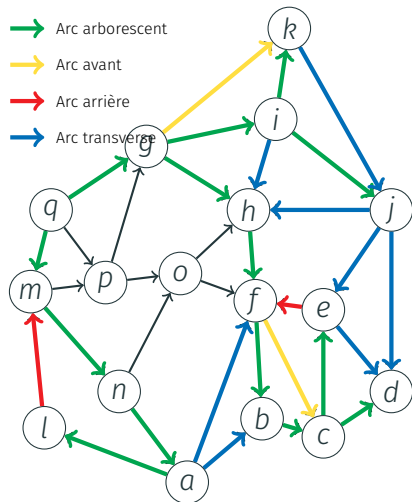


Classification de l'arborescence de parcours

Définition

Après un DFS, en notant (V, A) l'arborescence de parcours, on distingue les

- **arcs arborescents** : arcs de A
- **arcs avant** : uv où v est un descendant de u dans (V, A)
- **arcs arrière** : uv où u est un descendant de v dans (V, A)
- **arcs transverses** : les autres.

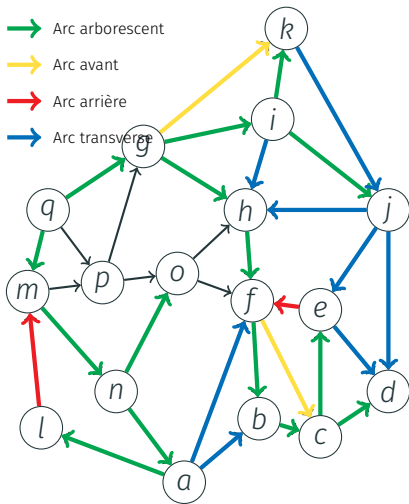


Classification de l'arborescence de parcours

Définition

Après un DFS, en notant (V, A) l'arborescence de parcours, on distingue les

- arcs arborescents : arcs de A
- arcs avant : uv où v est un descendant de u dans (V, A)
- arcs arrière : uv où u est un descendant de v dans (V, A)
- arcs transverses : les autres.

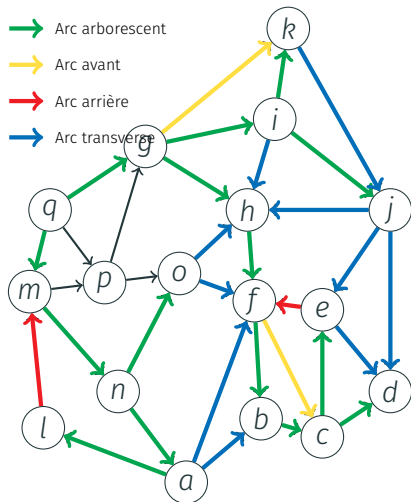


Classification de l'arborescence de parcours

Définition

Après un DFS, en notant (V, A) l'arborescence de parcours, on distingue les

- **arcs arborescents** : arcs de A
- **arcs avant** : uv où v est un descendant de u dans (V, A)
- **arcs arrière** : uv où u est un descendant de v dans (V, A)
- **arcs transverses** : les autres.

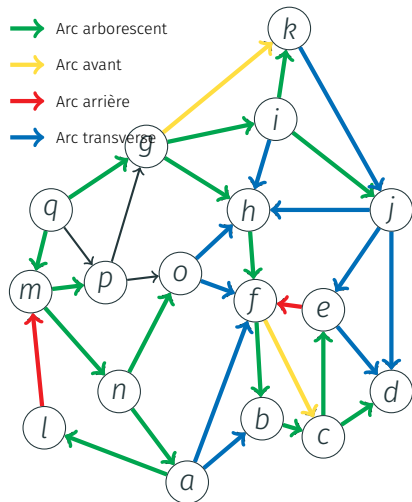


Classification de l'arborescence de parcours

Définition

Après un DFS, en notant (V, A) l'arborescence de parcours, on distingue les

- **arcs arborescents** : arcs de A
- **arcs avant** : uv où v est un descendant de u dans (V, A)
- **arcs arrière** : uv où u est un descendant de v dans (V, A)
- **arcs transverses** : les autres.

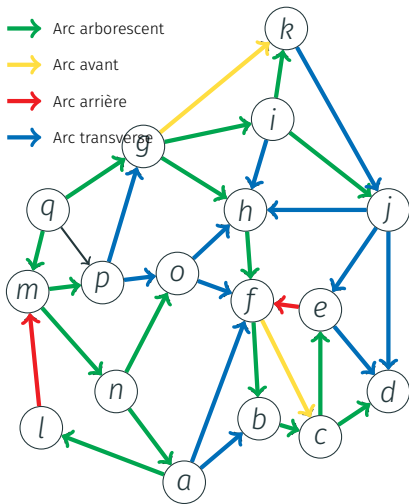


Classification de l'arborescence de parcours

Définition

Après un DFS, en notant (V, A) l'arborescence de parcours, on distingue les

- arcs arborescents : arcs de A
- arcs avant : uv où v est un descendant de u dans (V, A)
- arcs arrière : uv où u est un descendant de v dans (V, A)
- arcs transverses : les autres.

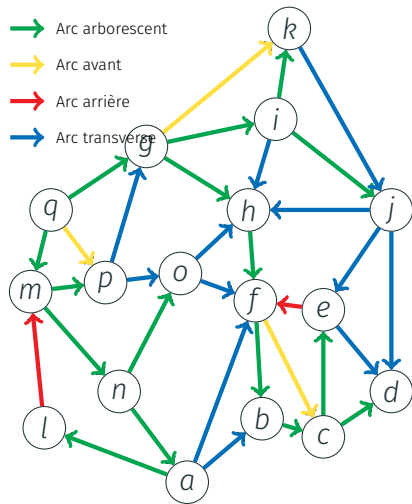


Classification de l'arborescence de parcours

Définition

Après un DFS, en notant (V, A) l'arborescence de parcours, on distingue les

- arcs arborescents : arcs de A
- arcs avant : uv où v est un descendant de u dans (V, A)
- arcs arrière : uv où u est un descendant de v dans (V, A)
- arcs transverses : les autres.



Théorème

*Un graphe orienté $G = (V, E)$ dont les sommets sont tous accessibles depuis s est **sans circuit** (ou acyclique) si et seulement si le parcours $DFS(G, s)$ ne rencontre pas d'arc arrière.*

Théorème

*Un graphe orienté $G = (V, E)$ dont les sommets sont tous accessibles depuis s est **sans circuit** (ou acyclique) si et seulement si le parcours $DFS(G, s)$ ne rencontre pas d'arc arrière.*

Démonstration.

Nous raisonnons sur les contraposées. Si $DFS(G, s)$ rencontre un arc arrière, disons l'arc uv , alors, d'une part u est accessible depuis v dans l'arborescence de parcours, donc il existe un chemin $v \rightsquigarrow u$. Mais, ce chemin peut être fermé avec l'arc uv pour former un cycle.

Théorème

*Un graphe orienté $G = (V, E)$ dont les sommets sont tous accessibles depuis s est **sans circuit** (ou acyclique) si et seulement si le parcours $DFS(G, s)$ ne rencontre pas d'arc arrière.*

Démonstration.

Nous raisonnons sur les contraposées. Si $DFS(G, s)$ rencontre un arc arrière, disons l'arc uv , alors, d'une part u est accessible depuis v dans l'arborescence de parcours, donc il existe un chemin $v \rightsquigarrow u$. Mais, ce chemin peut être fermé avec l'arc uv pour former un cycle.

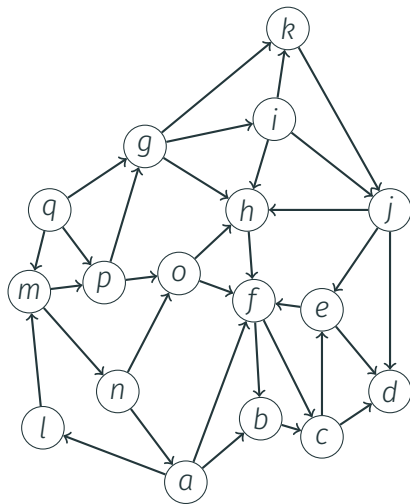
Réciproquement, supposons que G possède un cycle C . Notons $x_1, x_2, \dots, x_t$ les sommets de ce cycle, où x_1 est le premier sommet détecté par le parcours. Nous notons que le sommet x_t est accessible depuis x_1 . Donc $DFS(G, x_1)$ doit rencontrer x_t . De plus, quand x_t est rencontré, l'arc $x_t x_1$ sera testé et sera déclaré arc arrière. □

Détection de circuit : illustration

Définition

Après un DFS, en notant (V, A) l'arborescence de parcours, on distingue les

- **arcs arborescents** : arcs de A
- **arcs avant** : uv où v est un descendant de u dans (V, A)
- **arcs arrière** : uv où u est un descendant de v dans (V, A)
- **arcs transverses** : les autres.

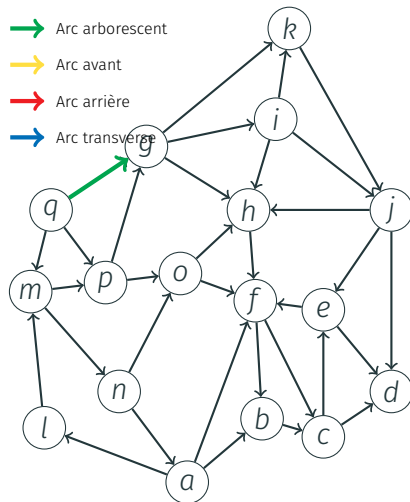


Détection de circuit : illustration

Définition

Après un DFS, en notant (V, A) l'arborescence de parcours, on distingue les

- **arcs arborescents** : arcs de A
- **arcs avant** : uv où v est un descendant de u dans (V, A)
- **arcs arrière** : uv où u est un descendant de v dans (V, A)
- **arcs transverses** : les autres.

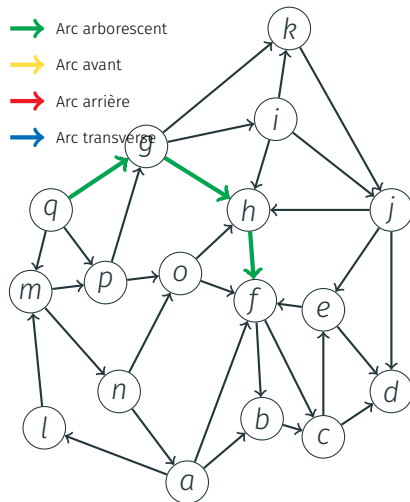


Détection de circuit : illustration

Définition

Après un DFS, en notant (V, A) l'arborescence de parcours, on distingue les

- **arcs arborescents** : arcs de A
- **arcs avant** : uv où v est un descendant de u dans (V, A)
- **arcs arrière** : uv où u est un descendant de v dans (V, A)
- **arcs transverses** : les autres.

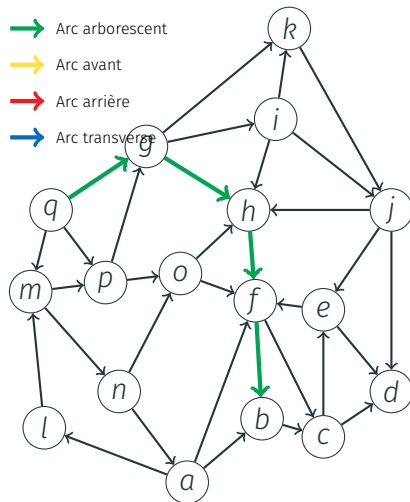


Détection de circuit : illustration

Définition

Après un DFS, en notant (V, A) l'arborescence de parcours, on distingue les

- **arcs arborescents** : arcs de A
- **arcs avant** : uv où v est un descendant de u dans (V, A)
- **arcs arrière** : uv où u est un descendant de v dans (V, A)
- **arcs transverses** : les autres.

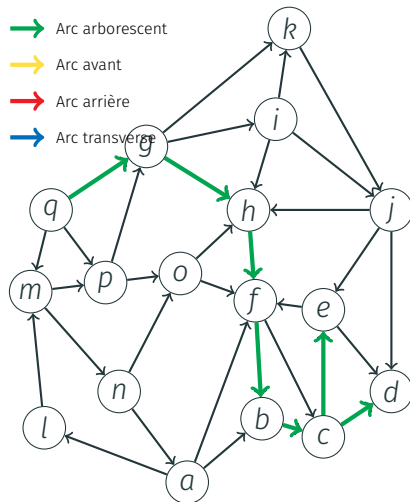


Détection de circuit : illustration

Définition

Après un DFS, en notant (V, A) l'arborescence de parcours, on distingue les

- **arcs arborescents** : arcs de A
- **arcs avant** : uv où v est un descendant de u dans (V, A)
- **arcs arrière** : uv où u est un descendant de v dans (V, A)
- **arcs transverses** : les autres.

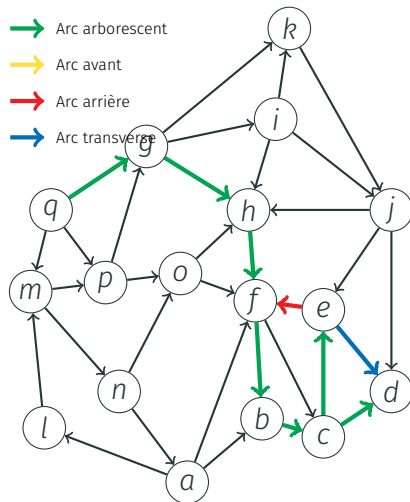


Détection de circuit : illustration

Définition

Après un DFS, en notant (V, A) l'arborescence de parcours, on distingue les

- **arcs arborescents** : arcs de A
- **arcs avant** : uv où v est un descendant de u dans (V, A)
- **arcs arrière** : uv où u est un descendant de v dans (V, A)
- **arcs transverses** : les autres.

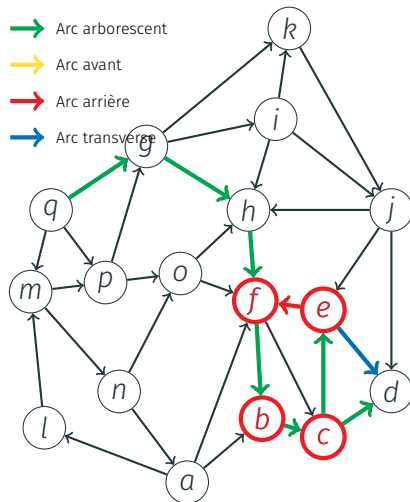


Détection de circuit : illustration

Définition

Après un DFS, en notant (V, A) l'arborescence de parcours, on distingue les

- **arcs arborescents** : arcs de A
- **arcs avant** : uv où v est un descendant de u dans (V, A)
- **arcs arrière** : uv où u est un descendant de v dans (V, A)
- **arcs transverses** : les autres.



Détection de cycle en python

```
class Graph():
    # [...]
    def hascycle(self, r, seen=None, done=None):
        if seen is None:
            seen = set({r})
            done = set()
        for u in self.neighbors[r]:
            if u not in seen:
                seen.add(u)
                if self.hascycle(u, seen, done):
                    return True
            elif u not in done:
                return True
        done.add(r)
        return False
```

Théorème

La relation d'accessibilité $\rightsquigarrow$ dans un graphe orienté est une relation d'ordre si et seulement si le graphe ne contient pas de circuit.

Accessibilité comme une relation d'ordre

Théorème

La relation d'accessibilité $\rightsquigarrow$ dans un graphe orienté est une relation d'ordre si et seulement si le graphe ne contient pas de circuit.

Démonstration.

Réflexivité et transitivité déjà vues.

Accessibilité comme une relation d'ordre

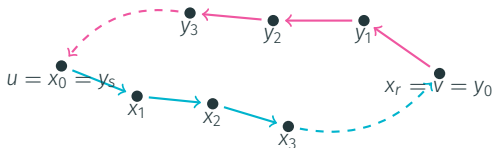
Théorème

La relation d'accessibilité $\rightsquigarrow$ dans un graphe orienté est une relation d'ordre si et seulement si le graphe ne contient pas de circuit.

Démonstration.

Réflexivité et transitivité déjà vues.

Antisymétrie : $(u \rightsquigarrow v) \wedge (v \rightsquigarrow u)$ équivaut à un circuit passant par u et v .



□

Tri topologique : définition

Définition

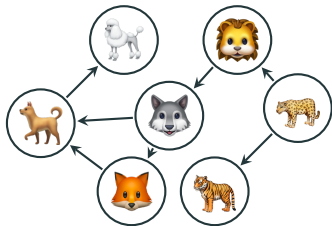
Un **tri topologique** des n sommets de $G = (V, E)$ est une numérotation $V = \{v_1, v_2, \dots, v_n\}$ telle que pour tout i et $j \in \llbracket 1, n \rrbracket$ distincts, si $v_i v_j$ est un arc de E , alors $i < j$.

Remarque

Mathématiquement, un **tri topologique** revient à raffiner l'ordre partiel $\rightsquigarrow$ en un ordre total.

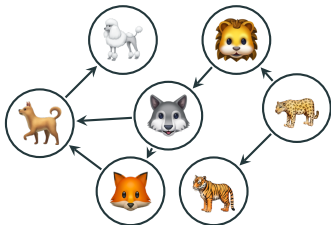
Tri topologique : illustration

Le graphe ci-dessous est orienté et sans circuit.



Tri topologique : illustration

Le graphe ci-dessous est orienté et sans circuit.



Il admet comme **numérotation topologique** l'ordre :

🐆, 🦁, 🐺, 🐅, 🦊, 🐕, 🐩.

Cet ordre n'est pas unique.

Tri topologique : existence et idée naïve

Lemme

Il existe toujours un sommet de degré entrant nul et un sommet de degré sortant nul.

Construire un ordre topologique en **itérant le lemme** n fois :
on stocke les degrés des sommets non numérotés dans un tableau,
quand on numérote un sommet, on diminue le degré des successeurs.

- Calcul des degrés entrants : $O(m)$
- Itération sur les sommets (extraction de la file, mise à jour des priorités) : $O(n + m)$.

1976



Robert Tarjan
(1948)

Algorithme 2 : Tri topologique

Entrées : Graphe $G(V, E)$

Sorties : Numérotation topologique des sommets

- 1 Initialiser un compteur
 - 2 **tant que** $\exists$ sommet v non numéroté **faire**
 - 3 | Lancer un DFS au départ de v
 - 4 | Attribuer le numéro postfixé aux sommets rencontrés
| (sans réinitialiser le compteur d'un DFS au suivant)
 - 5 Renverser l'ordre des numéros
-

1976



Robert Tarjan
(1948)

Algorithme 3 : Tri topologique

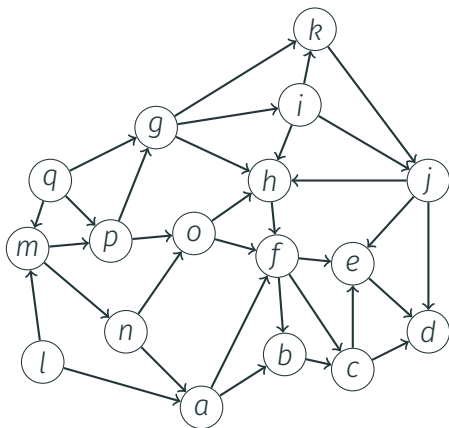
Entrées : Graphe $G(V, E)$

Sorties : Numérotation topologique des sommets

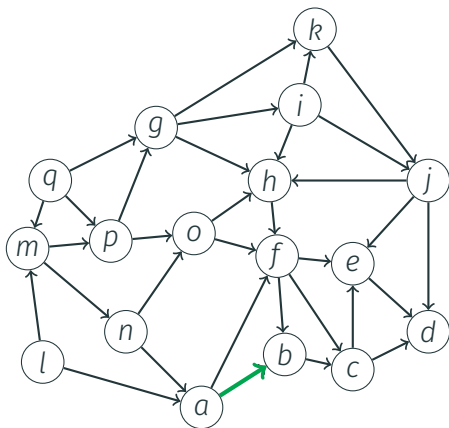
- 1 Initialiser un compteur
 - 2 **tant que** $\exists$ sommet v non numéroté **faire**
 - 3 | Lancer un DFS au départ de v
 - 4 | Attribuer le numéro postfixé aux sommets rencontrés
| (sans réinitialiser le compteur d'un DFS au suivant)
 - 5 Renverser l'ordre des numéros
-

Complexité : $O(n + m)$ (liste d'adjacence) ou $O(n^2)$ (matrice).

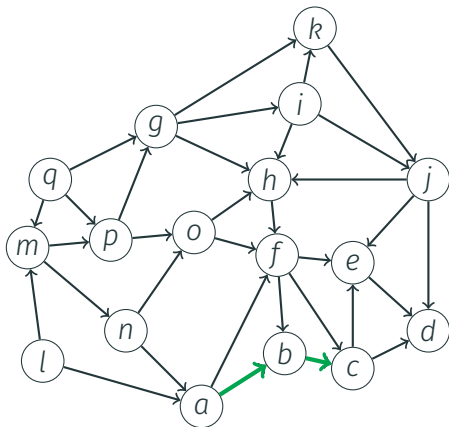
Tri topologique : illustration



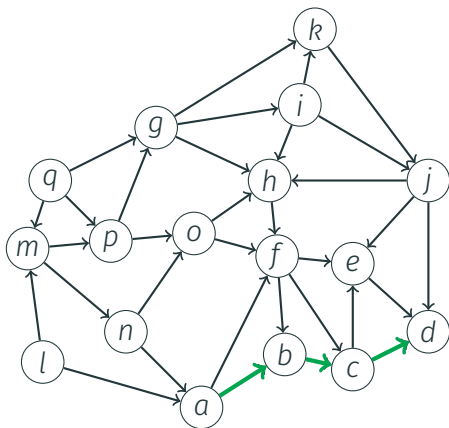
Tri topologique : illustration



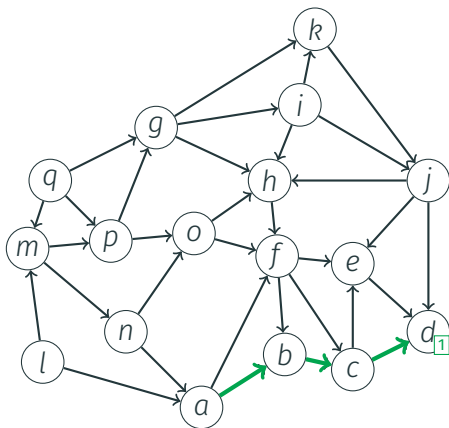
Tri topologique : illustration



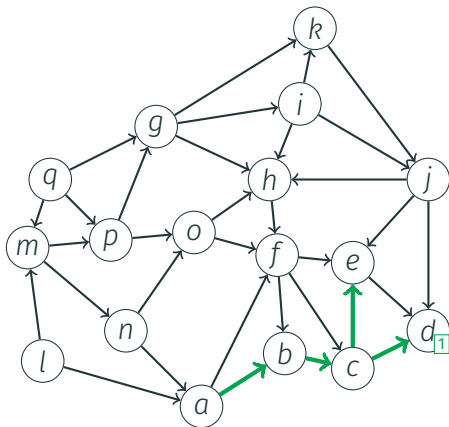
Tri topologique : illustration



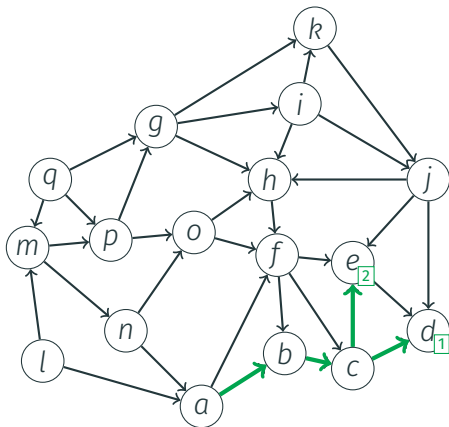
Tri topologique : illustration



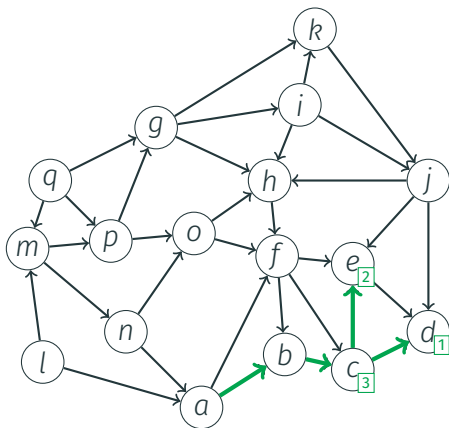
Tri topologique : illustration



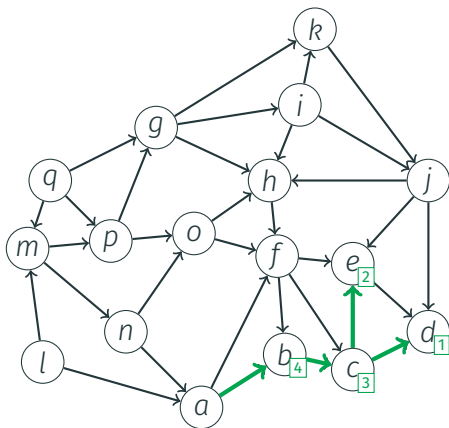
Tri topologique : illustration



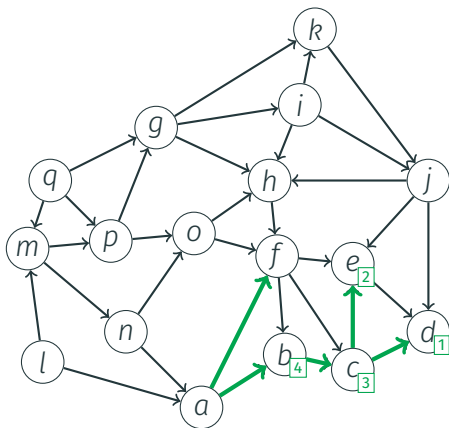
Tri topologique : illustration



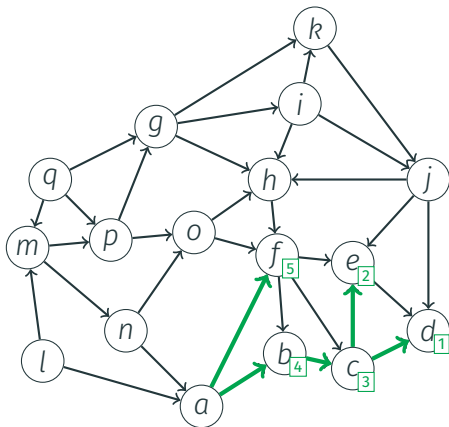
Tri topologique : illustration



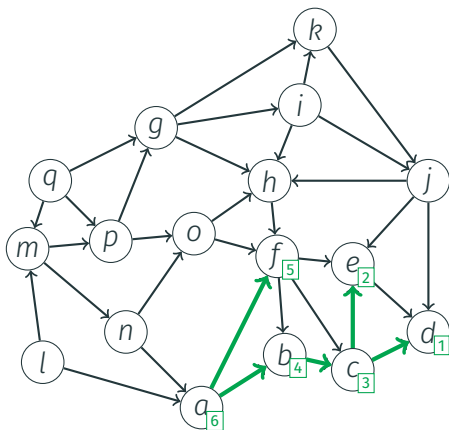
Tri topologique : illustration



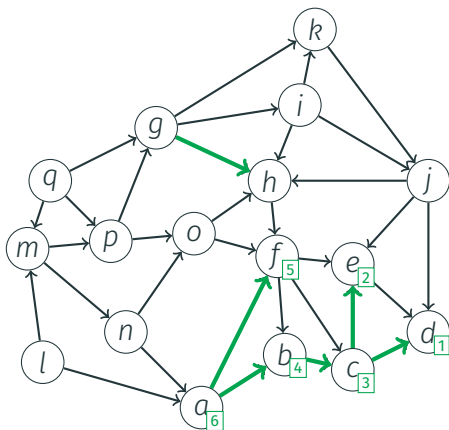
Tri topologique : illustration



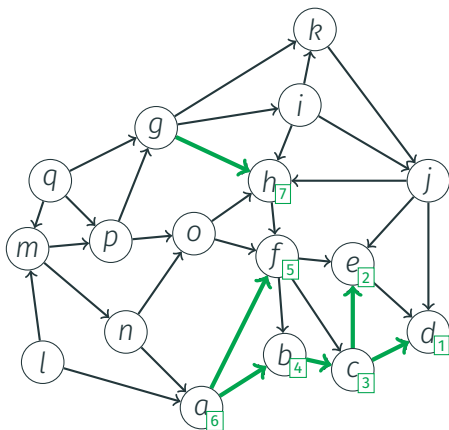
Tri topologique : illustration



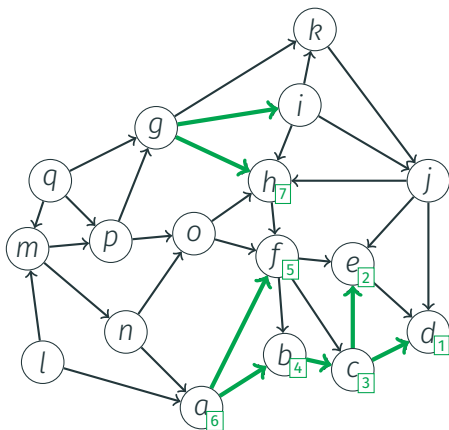
Tri topologique : illustration



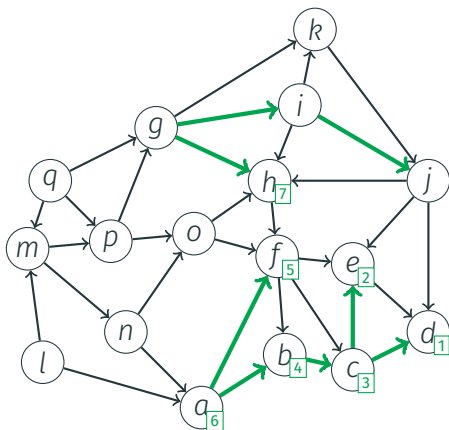
Tri topologique : illustration



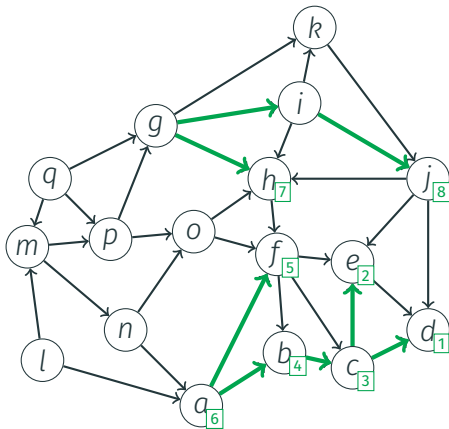
Tri topologique : illustration



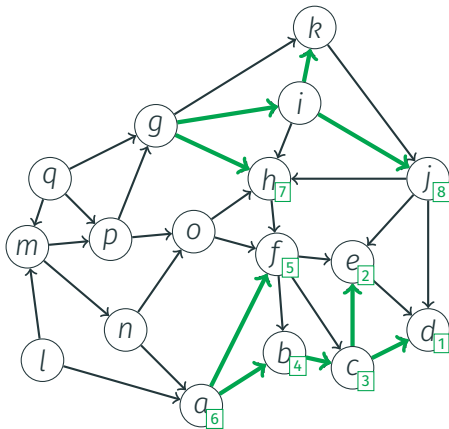
Tri topologique : illustration



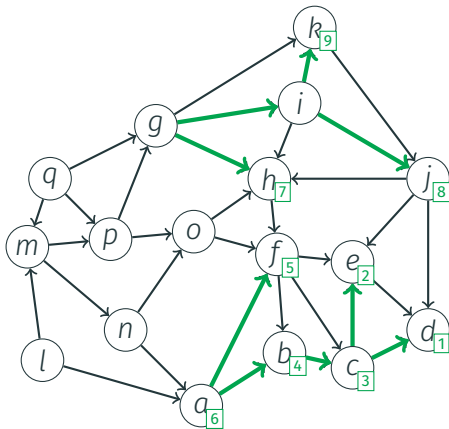
Tri topologique : illustration



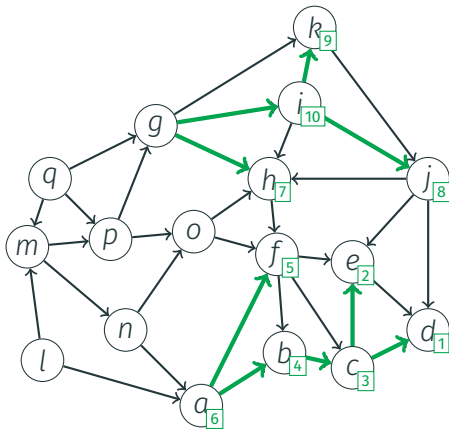
Tri topologique : illustration



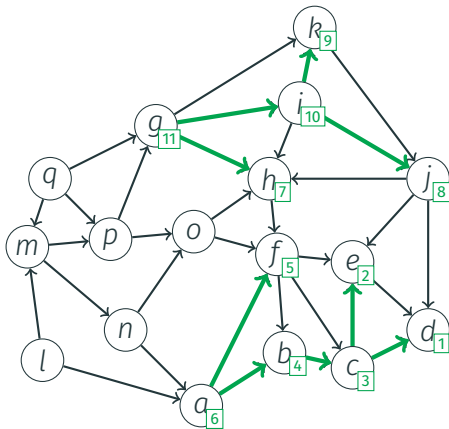
Tri topologique : illustration



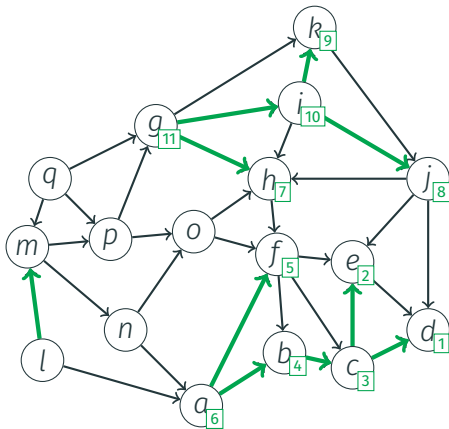
Tri topologique : illustration



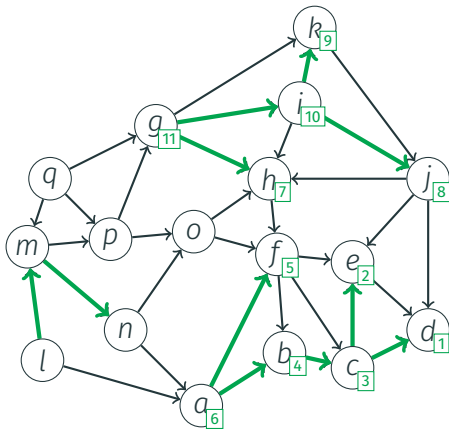
Tri topologique : illustration



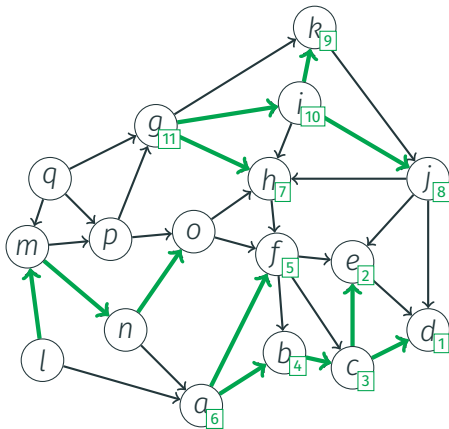
Tri topologique : illustration



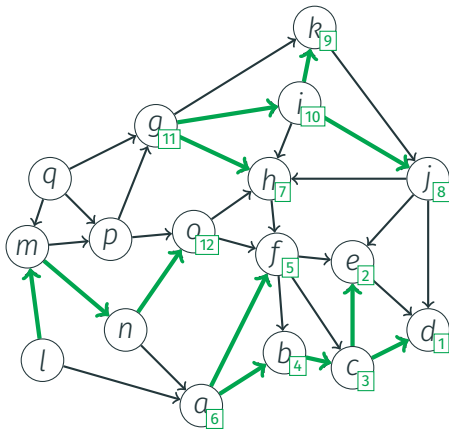
Tri topologique : illustration



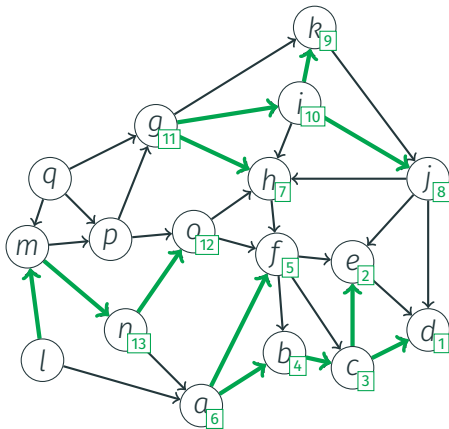
Tri topologique : illustration



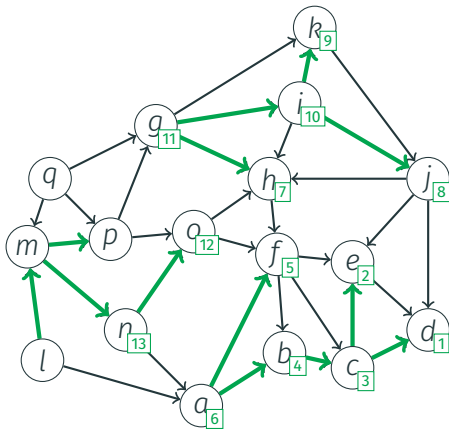
Tri topologique : illustration



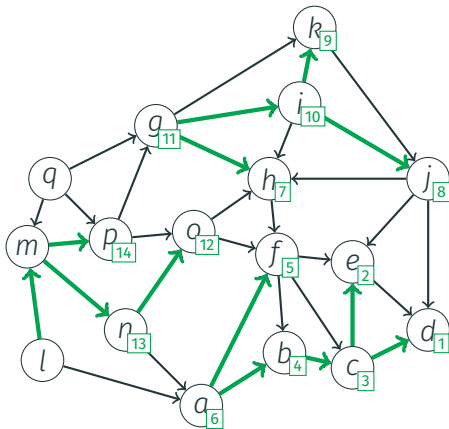
Tri topologique : illustration



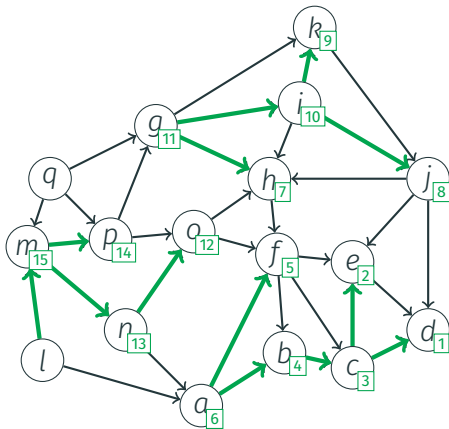
Tri topologique : illustration



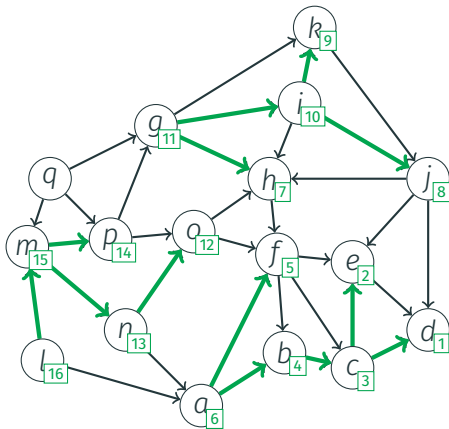
Tri topologique : illustration



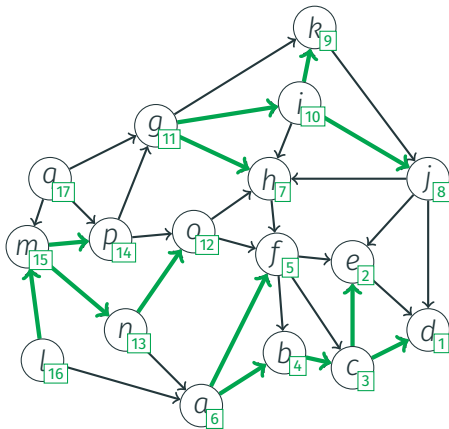
Tri topologique : illustration



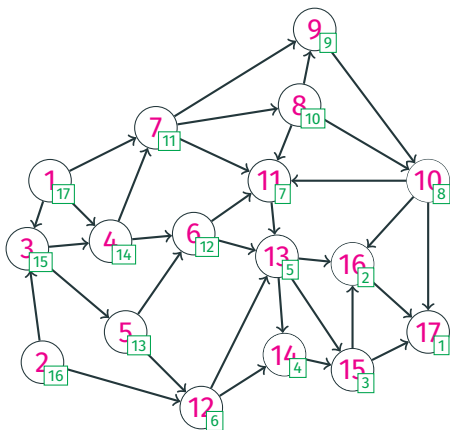
Tri topologique : illustration



Tri topologique : illustration



Tri topologique : illustration



Théorème

L'algorithme de Tarjan renvoie une numérotation topologique.

Preuve de correction (algo. de Tarjan)

Théorème

L'algorithme de Tarjan renvoie une numérotation topologique.

Démonstration.

Soient u et $v \in V$ tels que $u \rightsquigarrow v$, montrons que $\text{n}^\circ \text{post}(u) > \text{n}^\circ \text{post}(v)$.

Preuve de correction (algo. de Tarjan)

Théorème

L'algorithme de Tarjan renvoie une numérotation topologique.

Démonstration.

Soient u et $v \in V$ tels que $u \rightsquigarrow v$, montrons que $\text{n}^\circ \text{post}(u) > \text{n}^\circ \text{post}(v)$.

Raisonnons par l'absurde : supposons que $\text{n}^\circ \text{post}(u) < \text{n}^\circ \text{post}(v)$. Notons C un chemin de u vers v .

Preuve de correction (algo. de Tarjan)

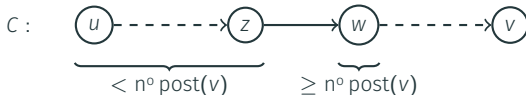
Théorème

L'algorithme de Tarjan renvoie une numérotation topologique.

Démonstration.

Soient u et $v \in V$ tels que $u \rightsquigarrow v$, montrons que $\text{n}^\circ \text{post}(u) > \text{n}^\circ \text{post}(v)$.

Raisonnons par l'absurde : supposons que $\text{n}^\circ \text{post}(u) < \text{n}^\circ \text{post}(v)$. Notons C un chemin de u vers v . Soit w le premier sommet de C tel que $\text{n}^\circ \text{post}(w) \geq \text{n}^\circ \text{post}(v)$ et z son prédécesseur.



Preuve de correction (algo. de Tarjan)

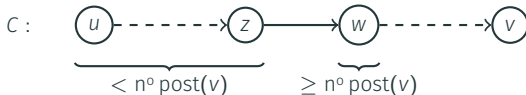
Théorème

L'algorithme de Tarjan renvoie une numérotation topologique.

Démonstration.

Soient u et $v \in V$ tels que $u \rightsquigarrow v$, montrons que $\text{n}^\circ \text{post}(u) > \text{n}^\circ \text{post}(v)$.

Raisonnons par l'absurde : supposons que $\text{n}^\circ \text{post}(u) < \text{n}^\circ \text{post}(v)$. Notons C un chemin de u vers v . Soit w le premier sommet de C tel que $\text{n}^\circ \text{post}(w) \geq \text{n}^\circ \text{post}(v)$ et z son prédécesseur.



On doit avoir $\text{n}^\circ \text{post}(z) < \text{n}^\circ \text{post}(v) \leq \text{n}^\circ \text{post}(w)$.

Preuve de correction (algo. de Tarjan)

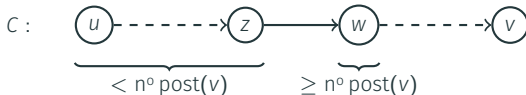
Théorème

L'algorithme de Tarjan renvoie une numérotation topologique.

Démonstration.

Soient u et $v \in V$ tels que $u \rightsquigarrow v$, montrons que $\text{n}^\circ \text{post}(u) > \text{n}^\circ \text{post}(v)$.

Raisonnons par l'absurde : supposons que $\text{n}^\circ \text{post}(u) < \text{n}^\circ \text{post}(v)$. Notons C un chemin de u vers v . Soit w le premier sommet de C tel que $\text{n}^\circ \text{post}(w) \geq \text{n}^\circ \text{post}(v)$ et z son prédécesseur.



On doit avoir $\text{n}^\circ \text{post}(z) < \text{n}^\circ \text{post}(v) \leq \text{n}^\circ \text{post}(w)$. Mais alors, comme $\text{n}^\circ \text{post}(z) < \text{n}^\circ \text{post}(w)$, lorsque le numéro post fixe est donné à z , l'algorithme n'a pas encore pu considérer w .

Preuve de correction (algo. de Tarjan)

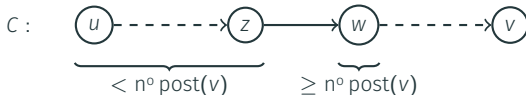
Théorème

L'algorithme de Tarjan renvoie une numérotation topologique.

Démonstration.

Soient u et $v \in V$ tels que $u \rightsquigarrow v$, montrons que $\text{n}^\circ \text{post}(u) > \text{n}^\circ \text{post}(v)$.

Raisonnons par l'absurde : supposons que $\text{n}^\circ \text{post}(u) < \text{n}^\circ \text{post}(v)$. Notons C un chemin de u vers v . Soit w le premier sommet de C tel que $\text{n}^\circ \text{post}(w) \geq \text{n}^\circ \text{post}(v)$ et z son prédécesseur.



On doit avoir $\text{n}^\circ \text{post}(z) < \text{n}^\circ \text{post}(v) \leq \text{n}^\circ \text{post}(w)$. Mais alors, comme $\text{n}^\circ \text{post}(z) < \text{n}^\circ \text{post}(w)$, lorsque le numéro post fixe est donné à z , l'algorithme n'a pas encore pu considérer w . Or w est un successeur de z et aurait dû être considéré dans l'appel $\text{DFS}(z)$ avant que le numéro z soit donné, ce qui est une contradiction! □

Forte connexité dans un graphe orienté

Soit $\longleftrightarrow$ la relation binaire de V

$$u \longleftrightarrow v \quad \text{si} \quad \begin{cases} u \rightsquigarrow v \\ v \rightsquigarrow u \end{cases}$$

Lemme

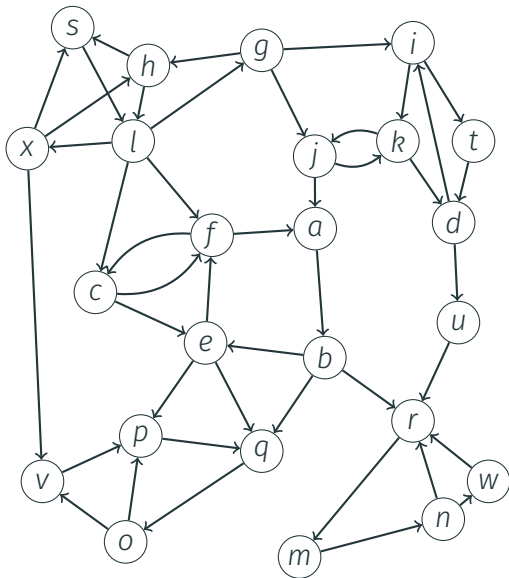
Dans un graphe orienté, $\longleftrightarrow$ est une *relation d'équivalence*.

Définition

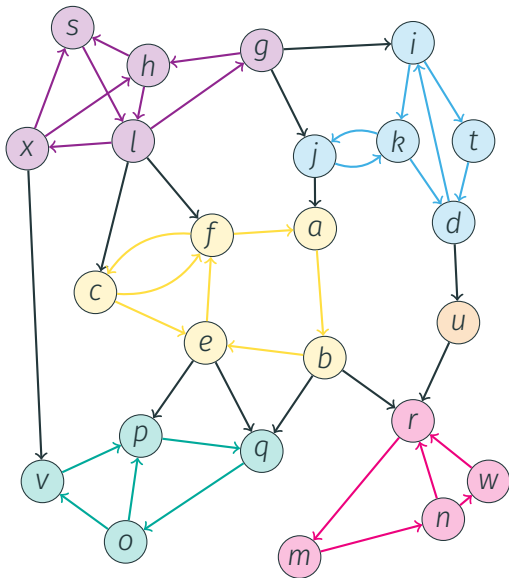
Les classes d'équivalence de $\longleftrightarrow$ s'appellent les *composantes fortement connexes*.

Un graphe est dit *fortement connexe* s'il ne possède qu'une seule composante connexe.

Exemple de composantes fortement connexes



Exemple de composantes fortement connexes



Graphe des composantes fortement connexes

On appelle **graphe des composantes fortement connexes** le graphe quotient sur $V / \rightsquigarrow$ ayant pour arcs

$$\bar{E} = \{(\bar{u}, \bar{v}); (u, v) \in E\}.$$

Théorème

Le graphe des composantes fortement connexes $(V / \rightsquigarrow, \bar{E})$ est un graphe orienté sans circuit.

Démonstration.

Un cycle C de $(V / \rightsquigarrow, \bar{E})$ peut se relever en un cycle du graphe initial G . Aussi, tous les sommets de C sont dans la même composante connexe de G et le cycle est trivial. □

Illustration du graphe des composantes fortement connexes

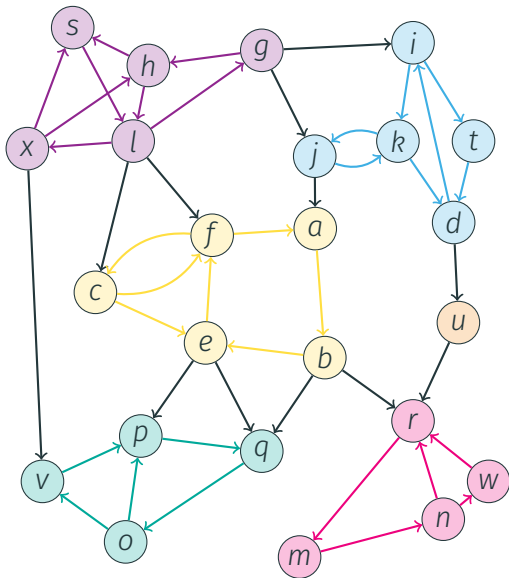


Illustration du graphe des composantes fortement connexes

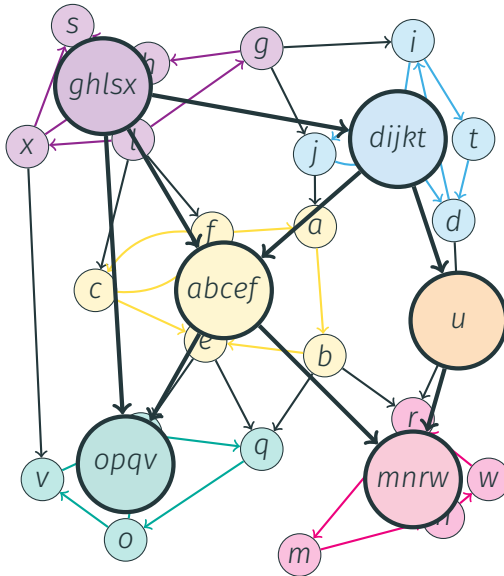
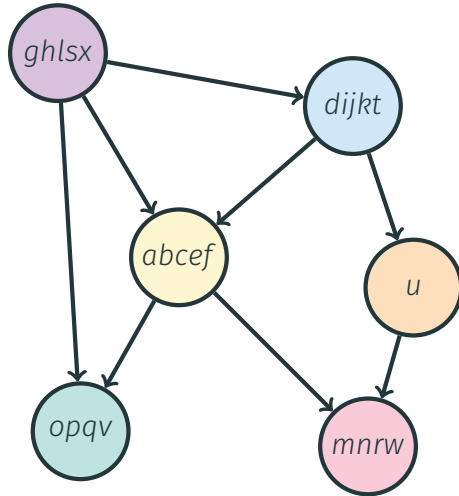


Illustration du graphe des composantes fortement connexes



Définition

Le **graphe transposé** d'un graphe $G = (V, E)$ est le graphe $G^T = (V, E^T)$ où

$$E^T = \{(v, u); (u, v) \in E\}.$$

Algorithme de Kosaraju-Sharir

Définition

Le **graphe transposé** d'un graphe $G = (V, E)$ est le graphe $G^T = (V, E^T)$ où

$$E^T = \{(v, u); (u, v) \in E\}.$$

~1978



S. Rao Kosaraju

Algorithme 5 : Algorithme de Kosaraju - Sharir

Entrées : Graphe orienté $G = (V, E)$

Résultat : Composantes fortement connexes

- 1 **tant que** $\exists$ sommet non numéroté **faire**
- 2 ┌ Lancer un DFS depuis un sommet quelconque et
 └ numéroté les sommets en ordre postfixe.
- 3 **tant que** $\exists$ sommet dans G^T **faire**
- 4 ┌ Lancer un DFS dans G^T depuis le sommet de plus haut
 └ numéro et supprimer les sommets visités de G^T .
- 5 Chaque DFS de la phase 2 fournit une composante.

Illustration de la phase 1

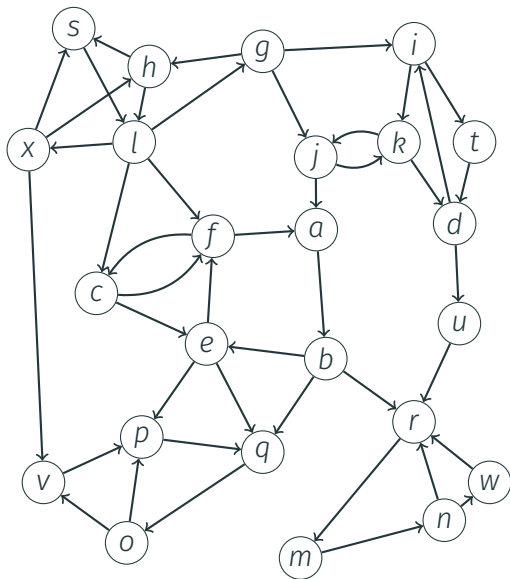


Illustration de la phase 1

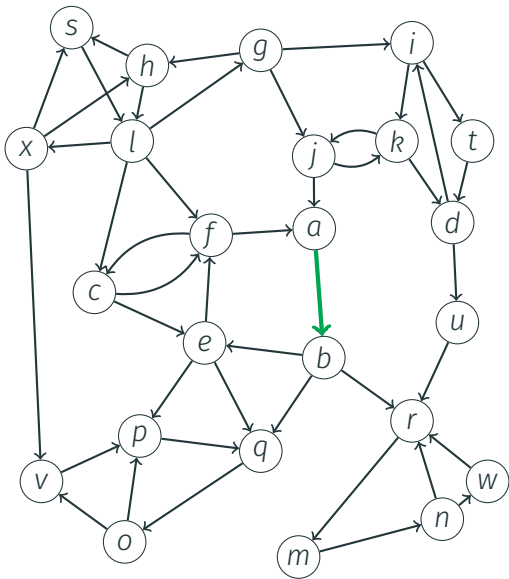


Illustration de la phase 1

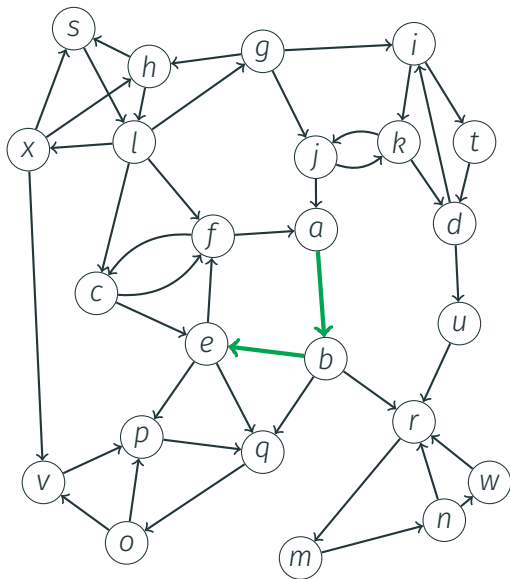


Illustration de la phase 1

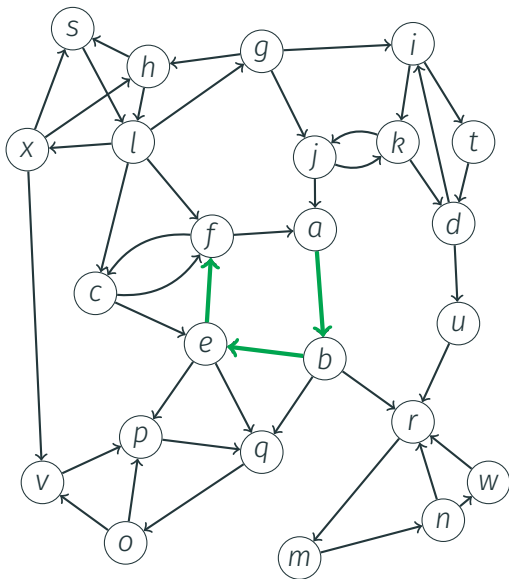


Illustration de la phase 1

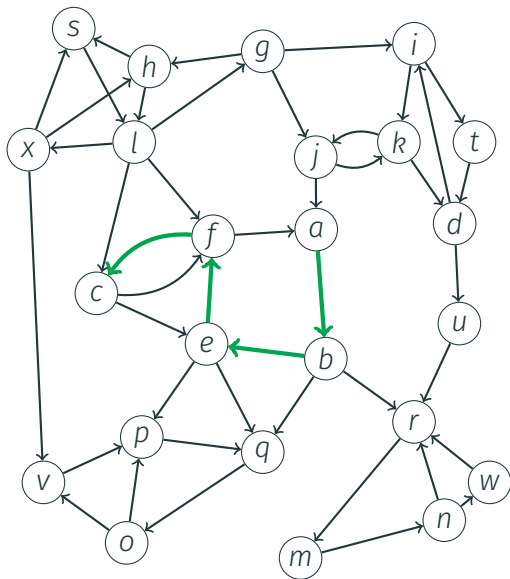


Illustration de la phase 1

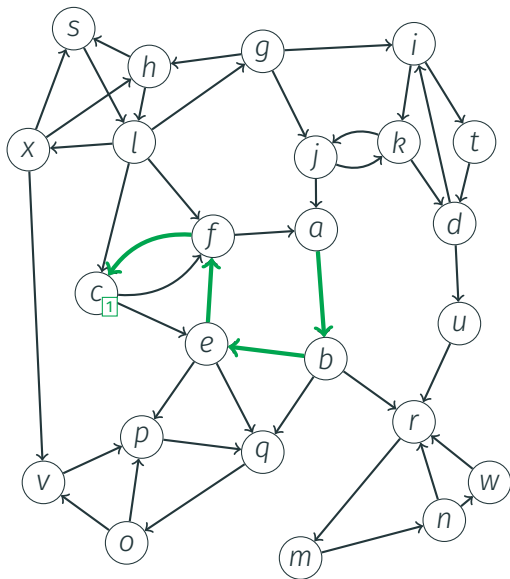


Illustration de la phase 1

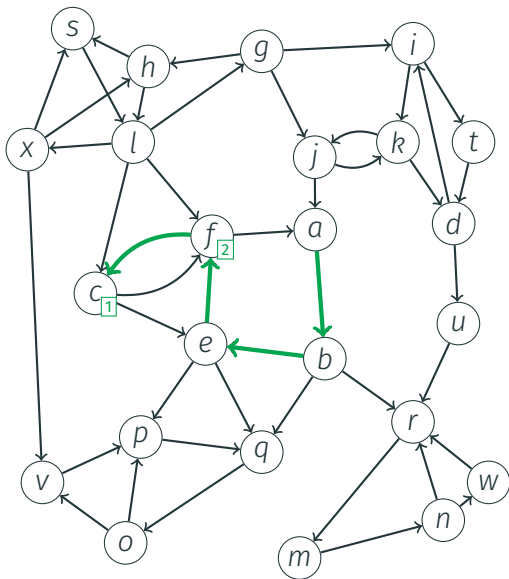


Illustration de la phase 1

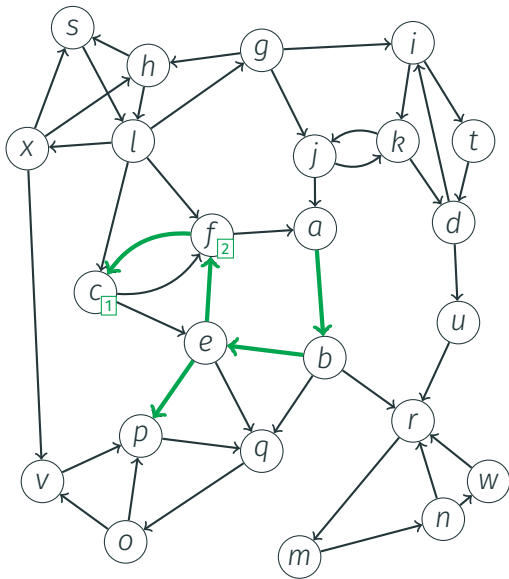


Illustration de la phase 1

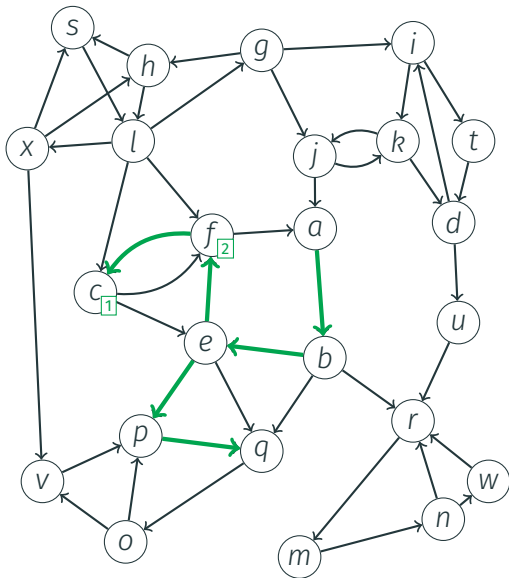


Illustration de la phase 1

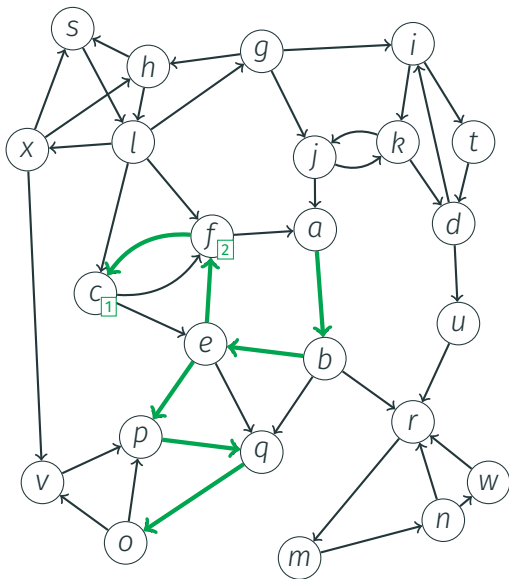


Illustration de la phase 1

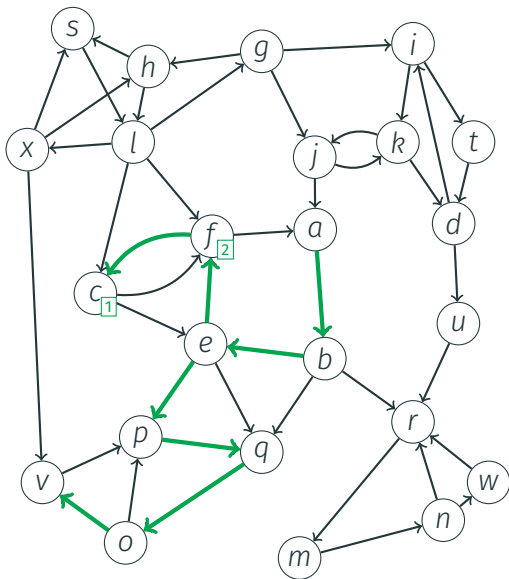


Illustration de la phase 1

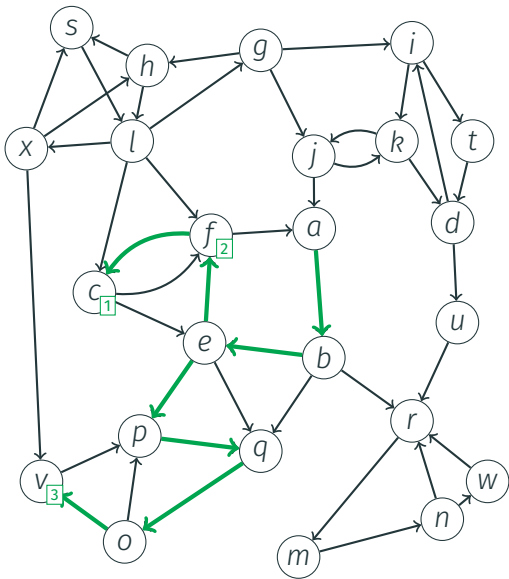


Illustration de la phase 1

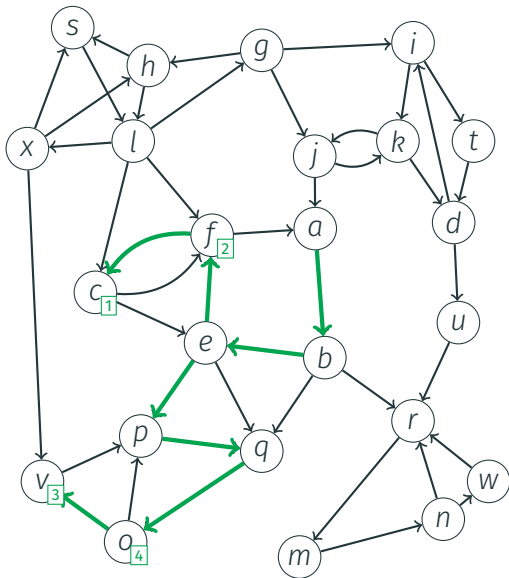


Illustration de la phase 1

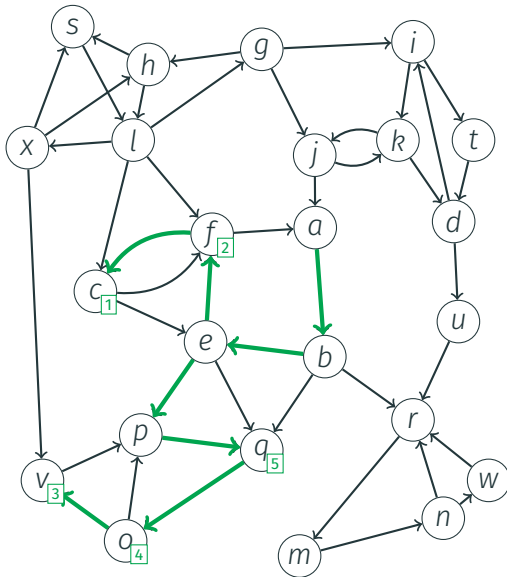


Illustration de la phase 1

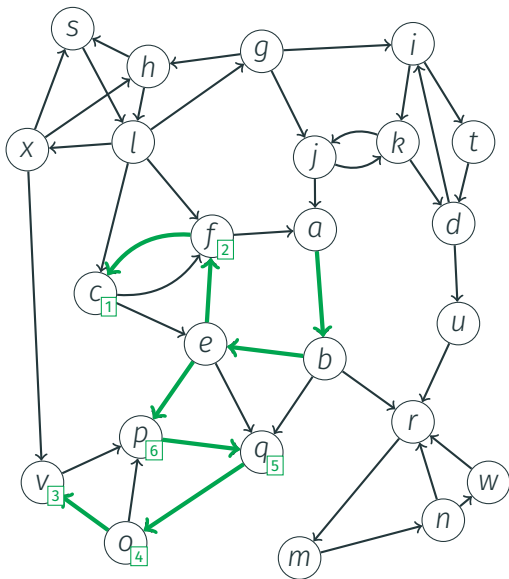


Illustration de la phase 1

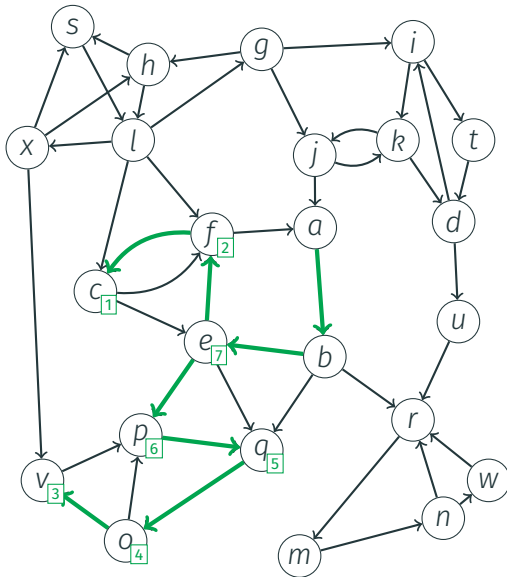


Illustration de la phase 1

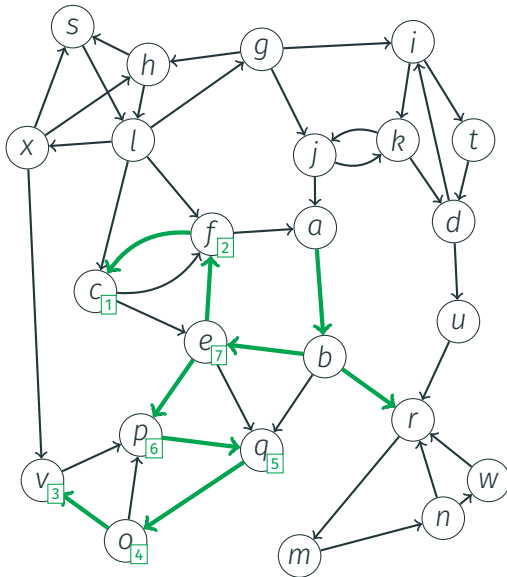


Illustration de la phase 1

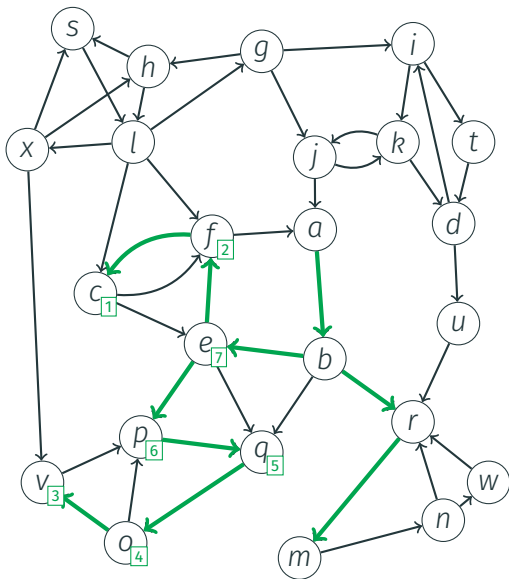


Illustration de la phase 1

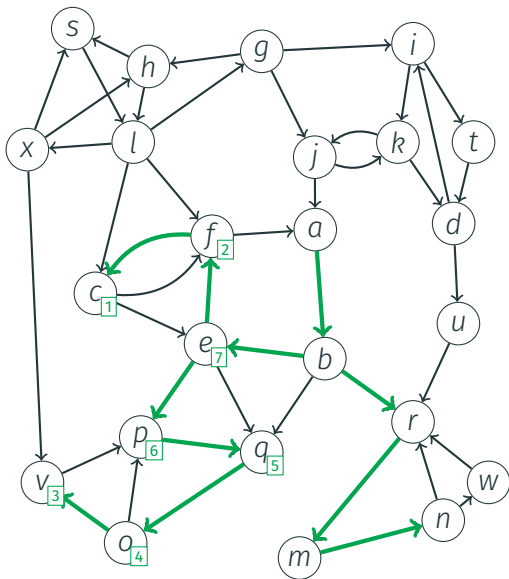


Illustration de la phase 1

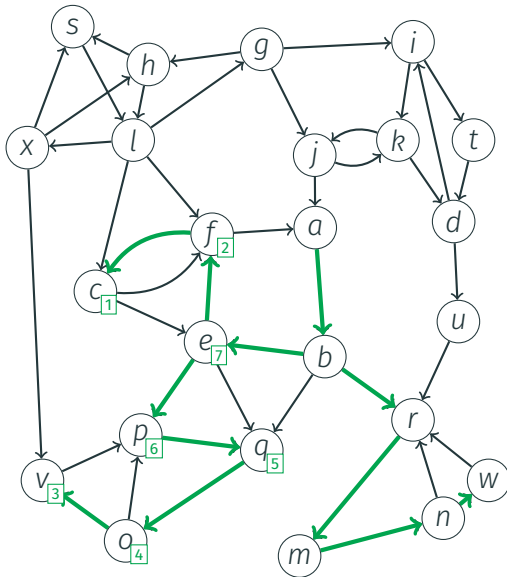


Illustration de la phase 1

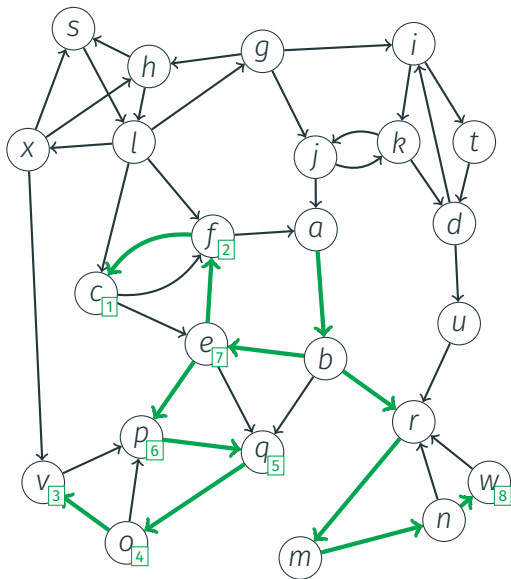


Illustration de la phase 1

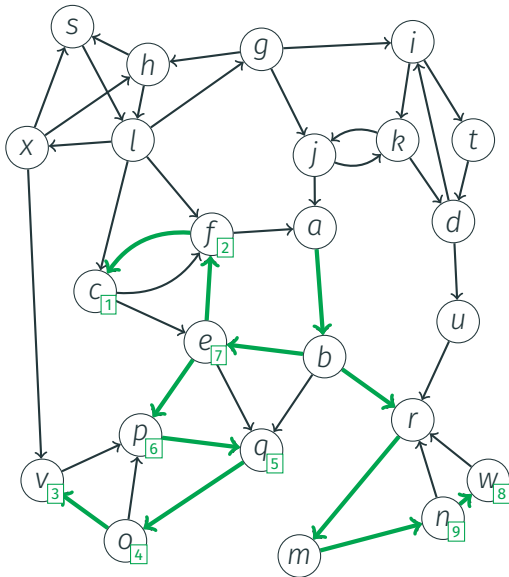


Illustration de la phase 1

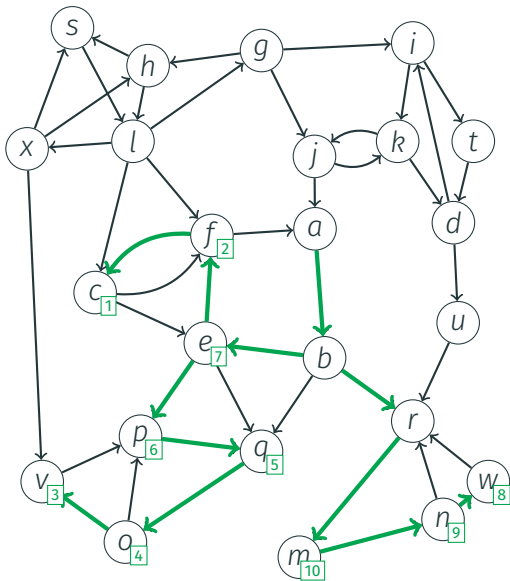


Illustration de la phase 1

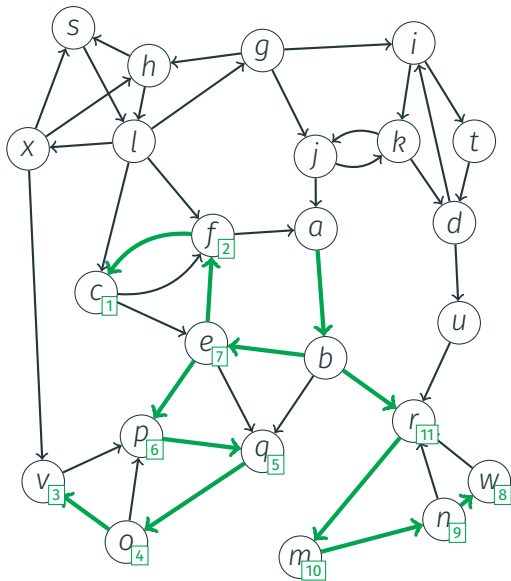


Illustration de la phase 1

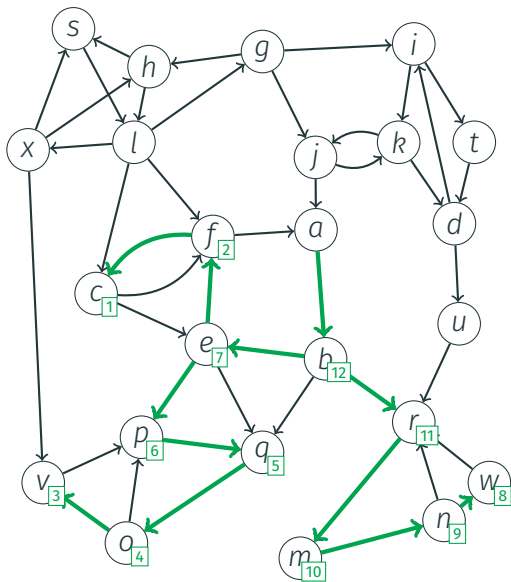


Illustration de la phase 1

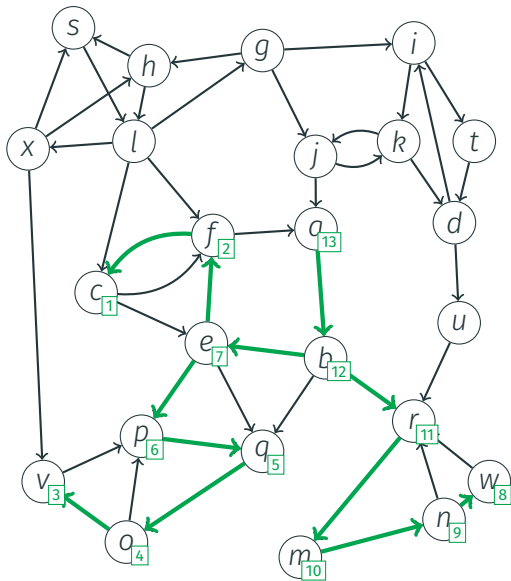


Illustration de la phase 1

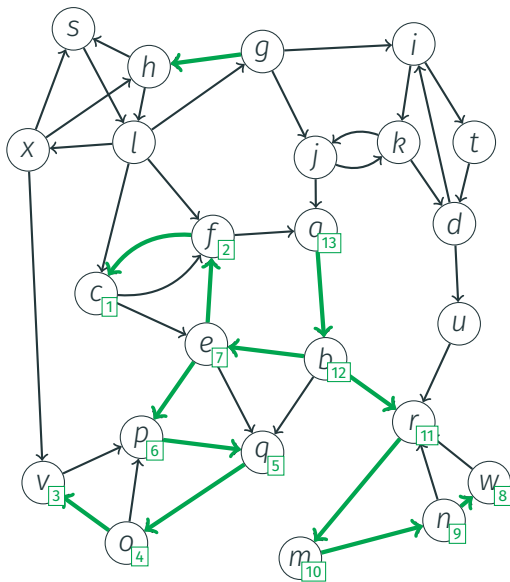


Illustration de la phase 1

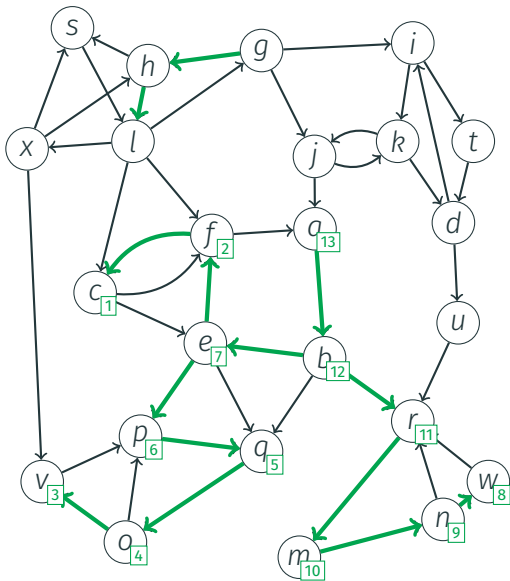


Illustration de la phase 1

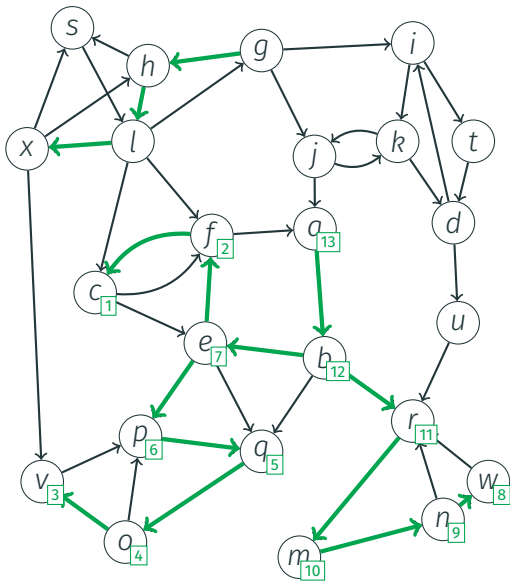


Illustration de la phase 1

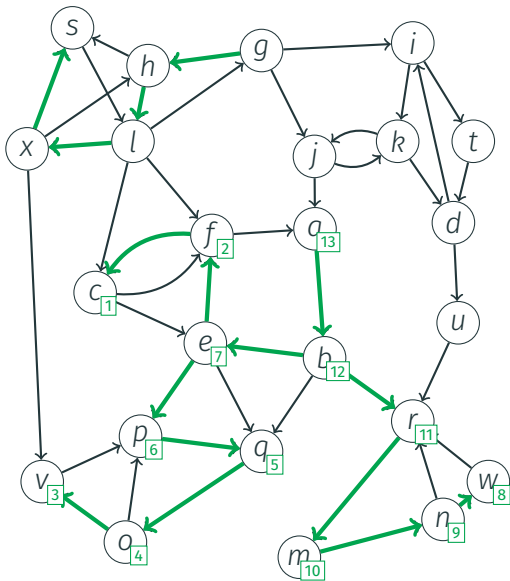


Illustration de la phase 1

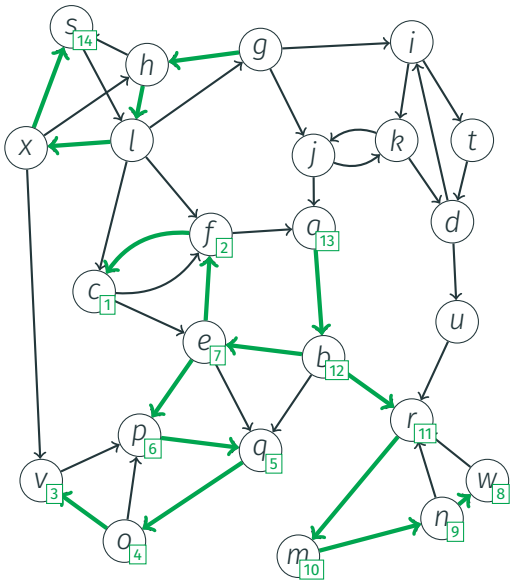


Illustration de la phase 1

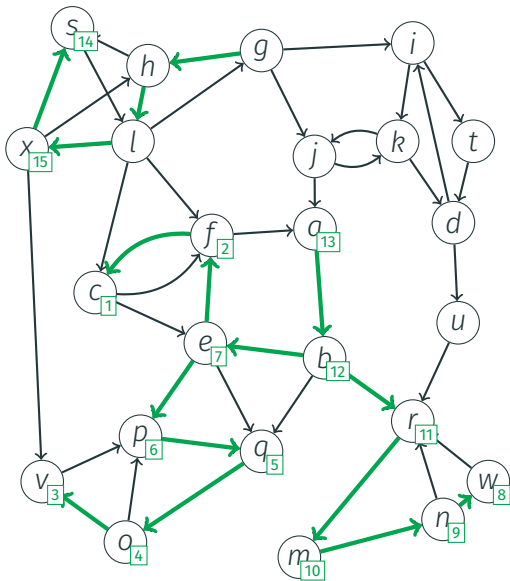


Illustration de la phase 1

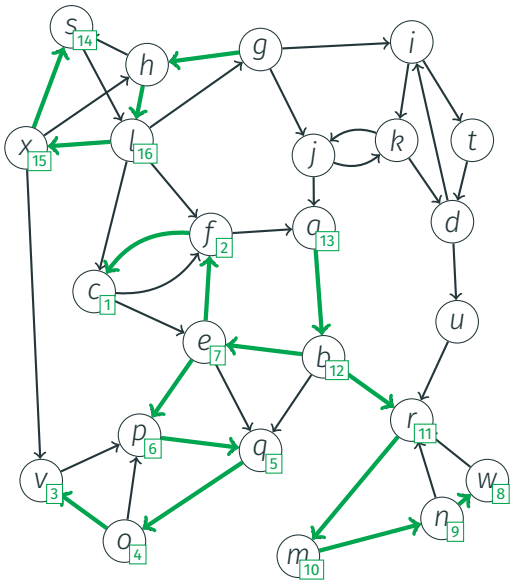


Illustration de la phase 1

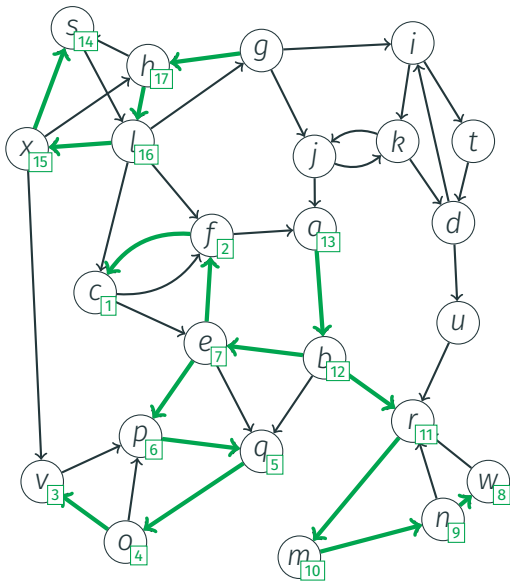


Illustration de la phase 1

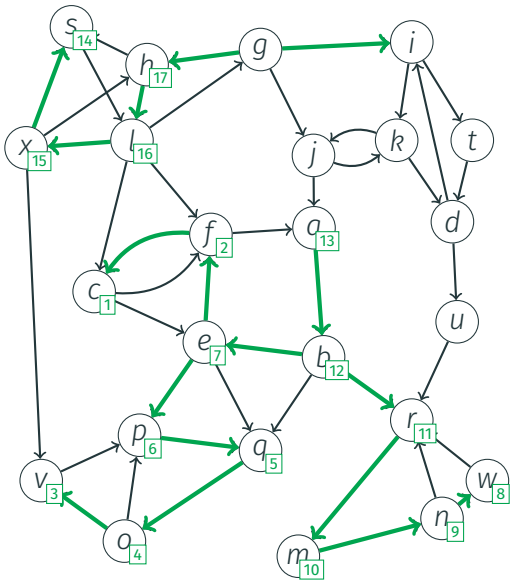


Illustration de la phase 1

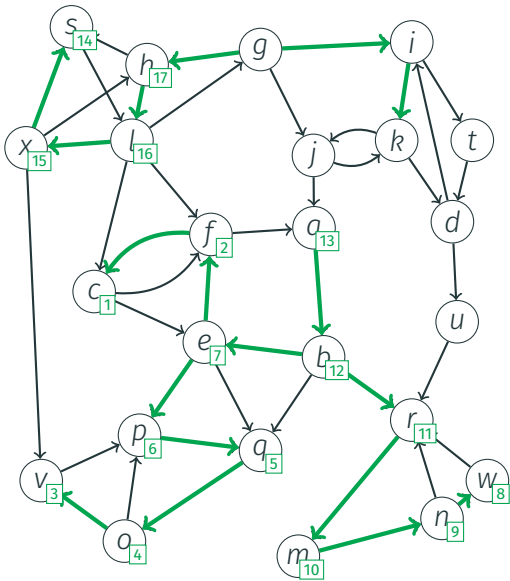


Illustration de la phase 1

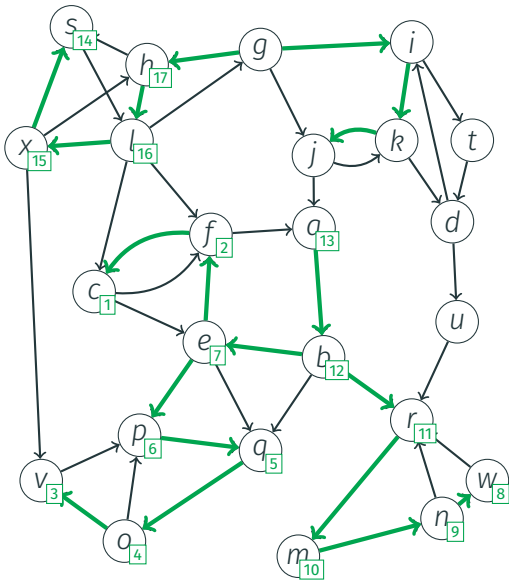


Illustration de la phase 1

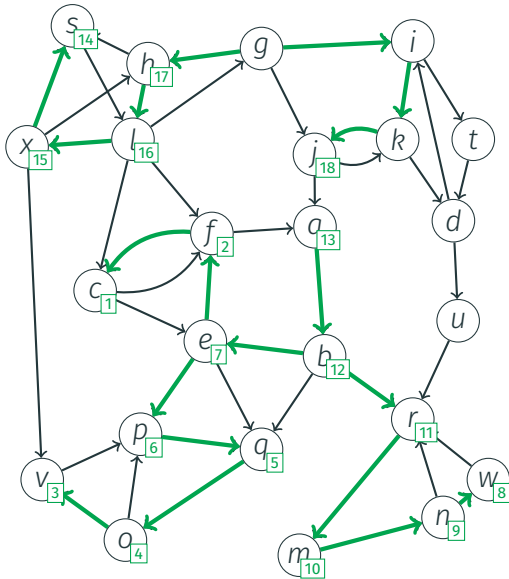


Illustration de la phase 1

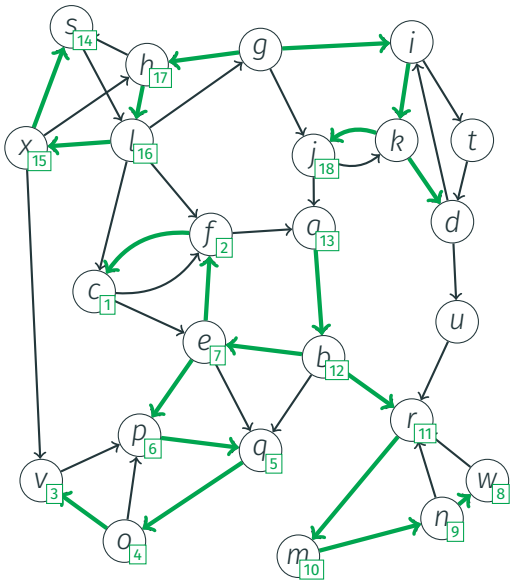


Illustration de la phase 1

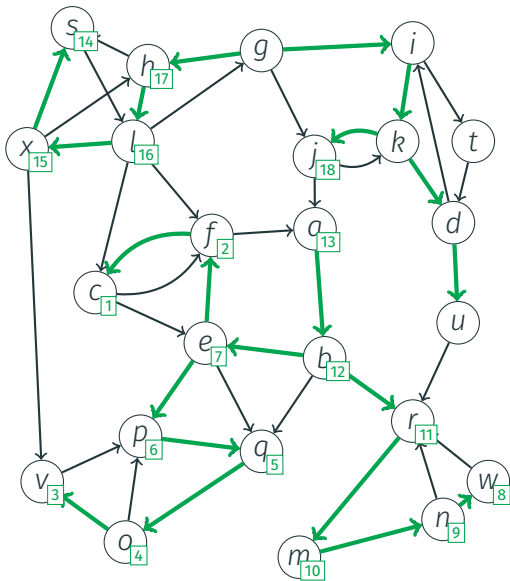


Illustration de la phase 1

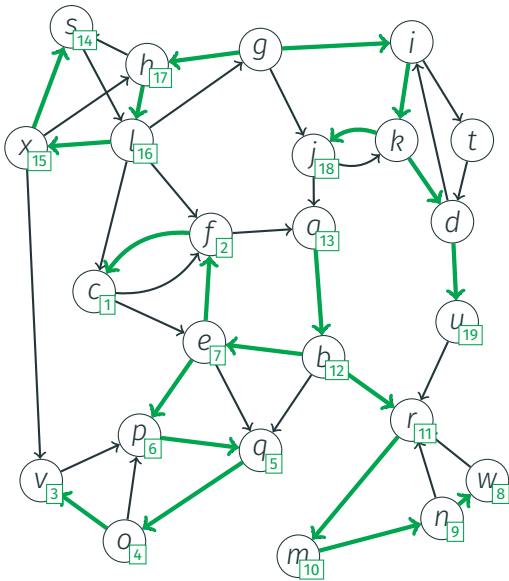


Illustration de la phase 1

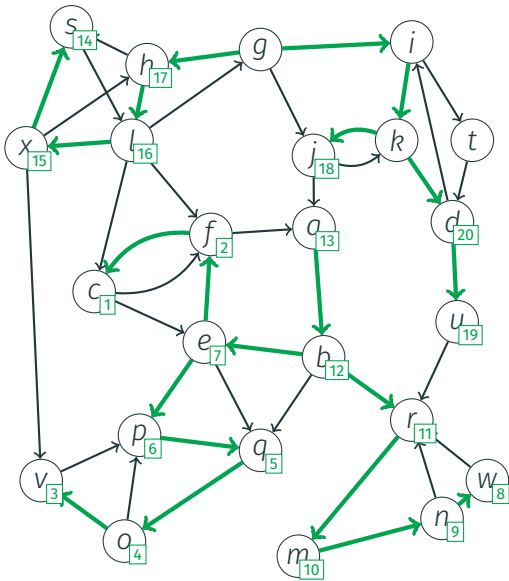


Illustration de la phase 1

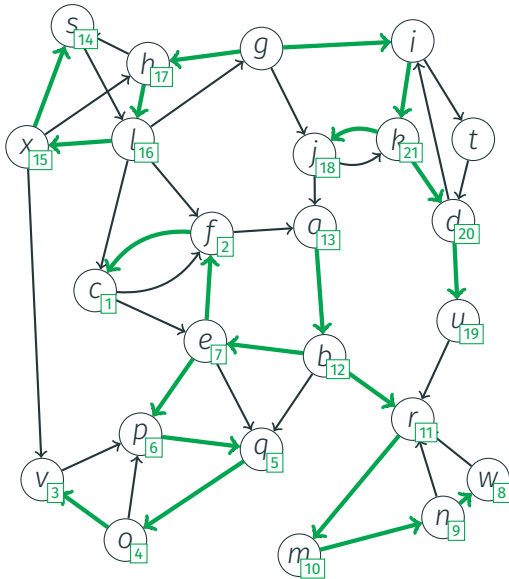


Illustration de la phase 1

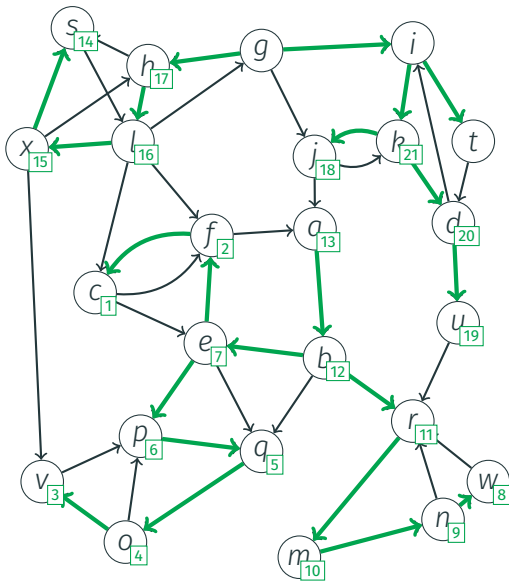


Illustration de la phase 1

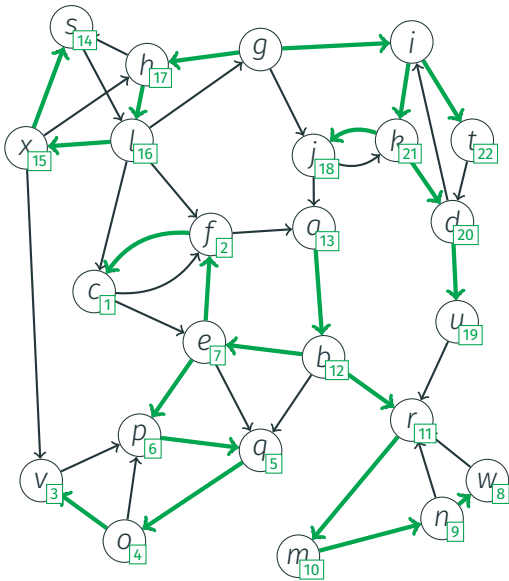


Illustration de la phase 1

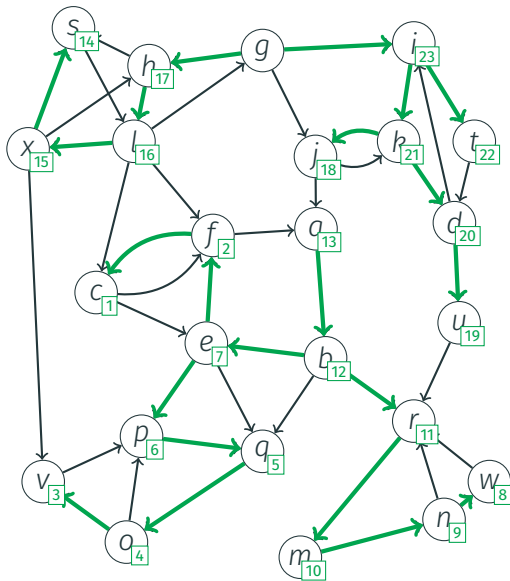


Illustration de la phase 1

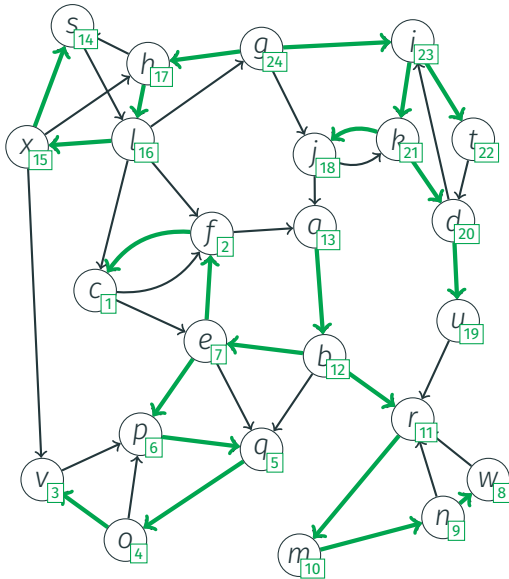


Illustration de la phase 2

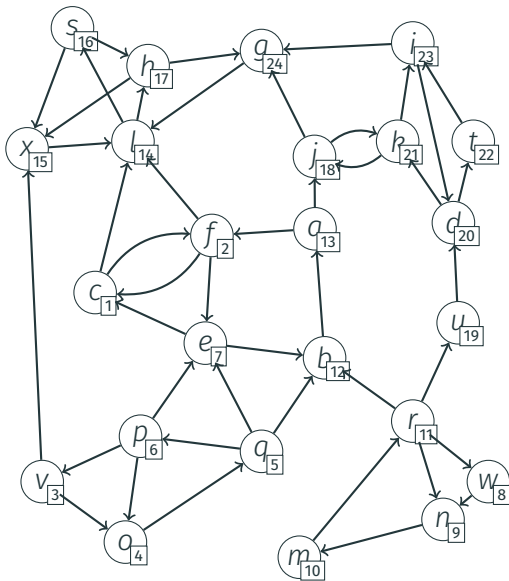


Illustration de la phase 2

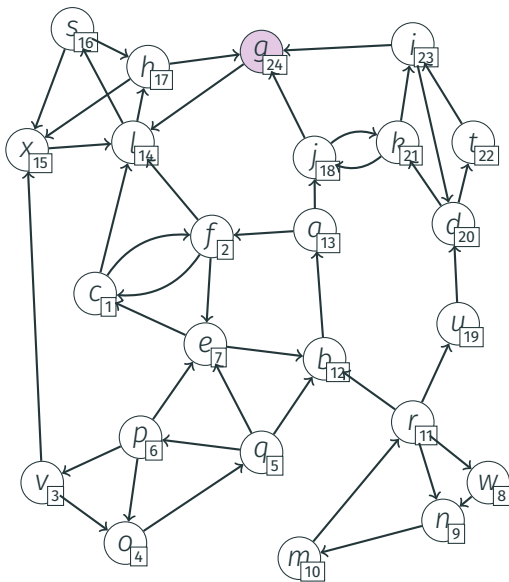


Illustration de la phase 2

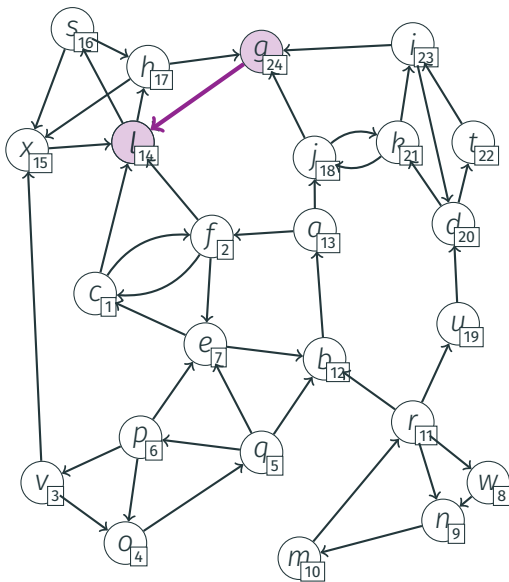


Illustration de la phase 2

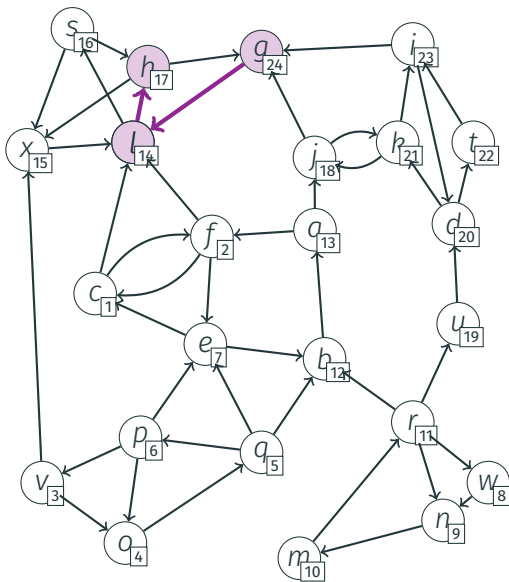


Illustration de la phase 2

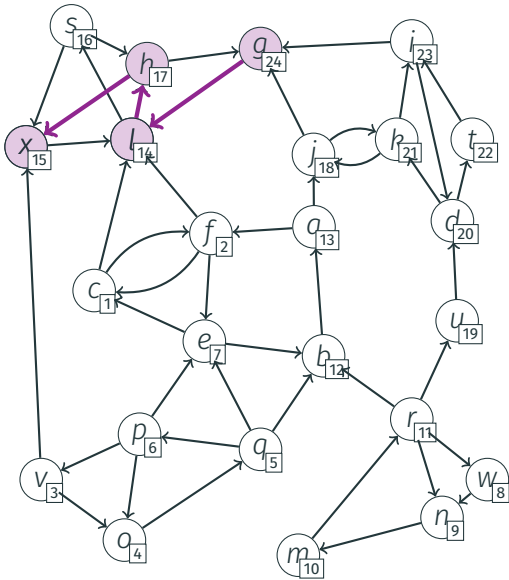


Illustration de la phase 2

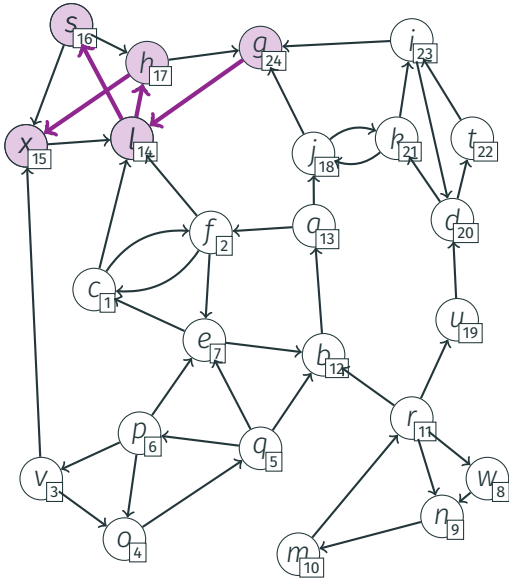


Illustration de la phase 2

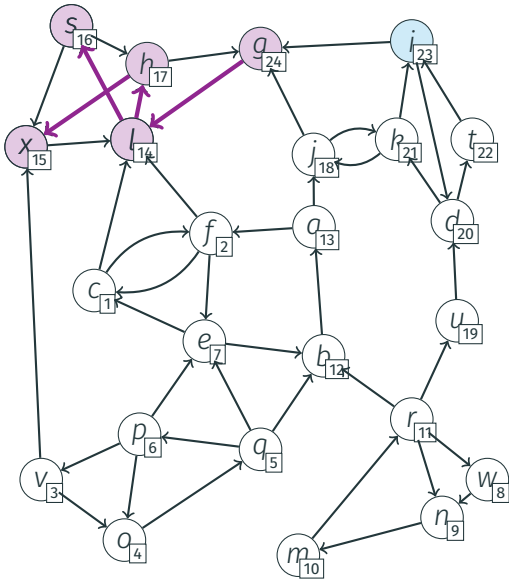


Illustration de la phase 2

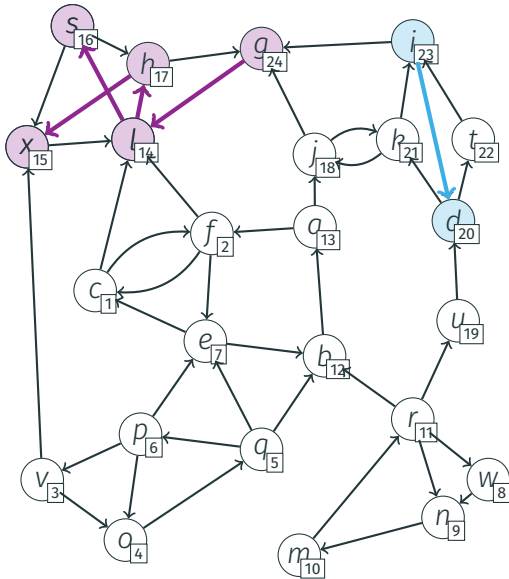


Illustration de la phase 2

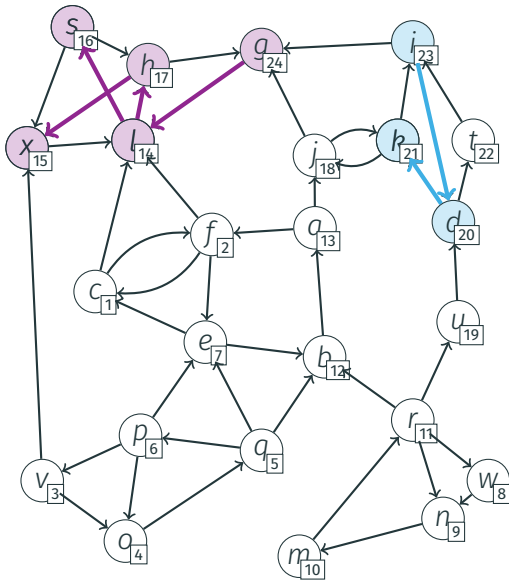


Illustration de la phase 2

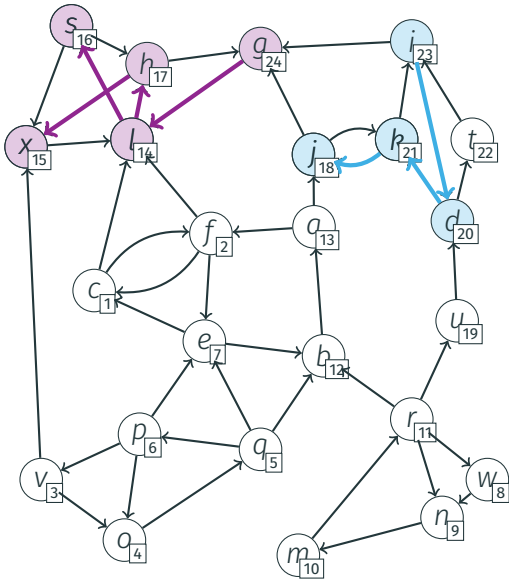


Illustration de la phase 2

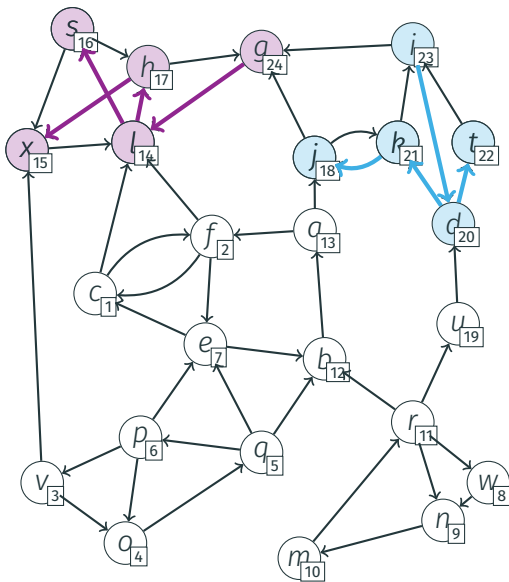


Illustration de la phase 2

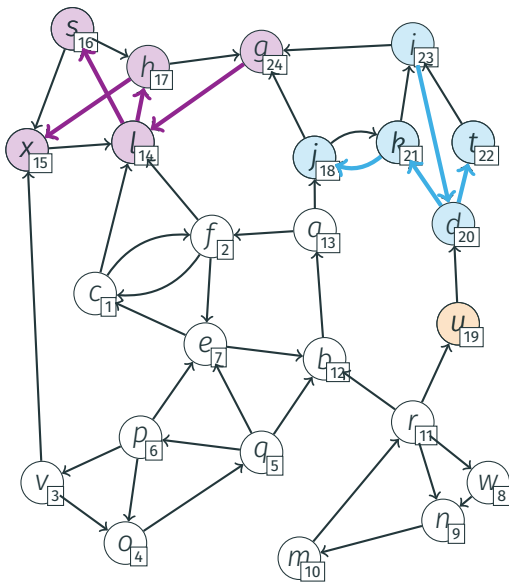


Illustration de la phase 2

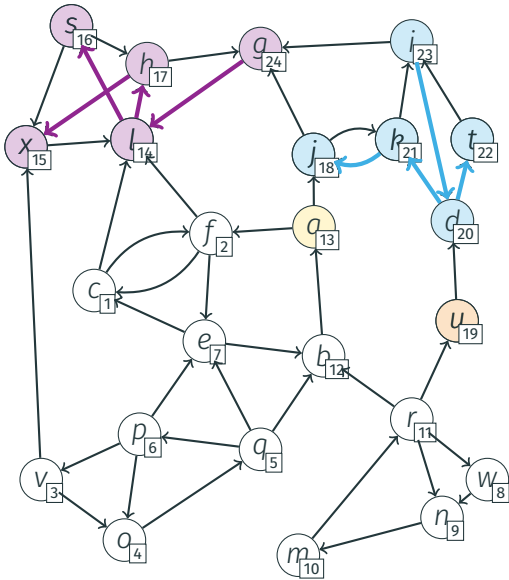


Illustration de la phase 2

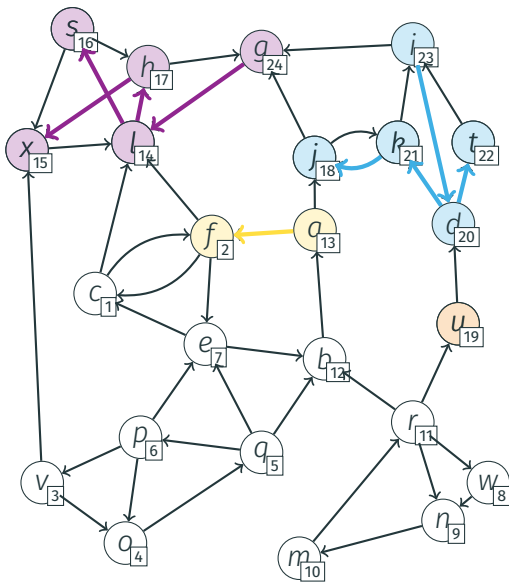


Illustration de la phase 2

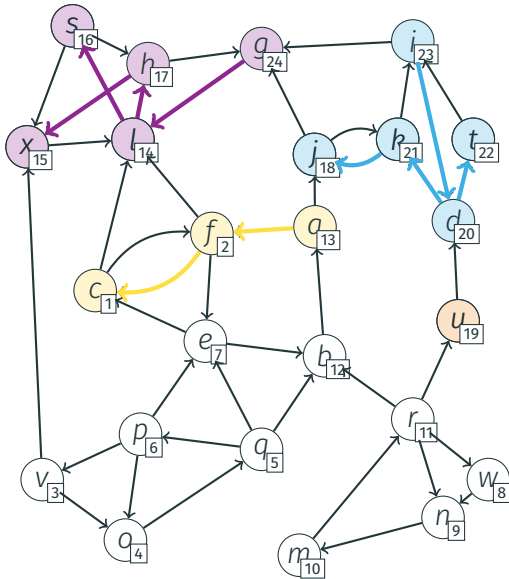


Illustration de la phase 2

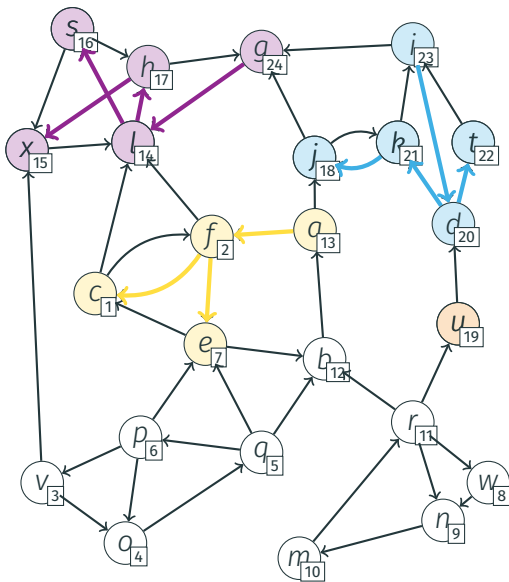


Illustration de la phase 2

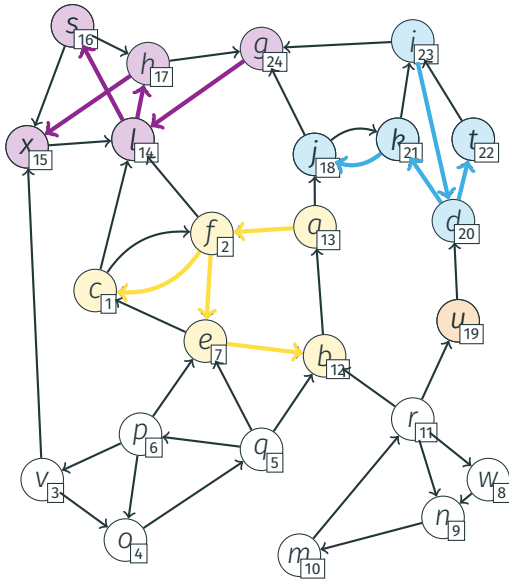


Illustration de la phase 2

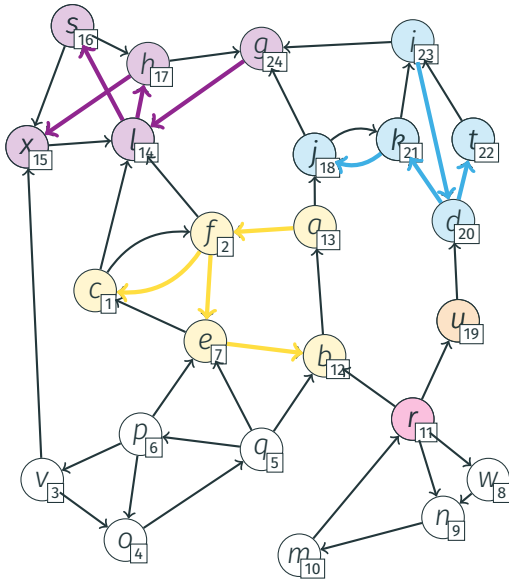


Illustration de la phase 2

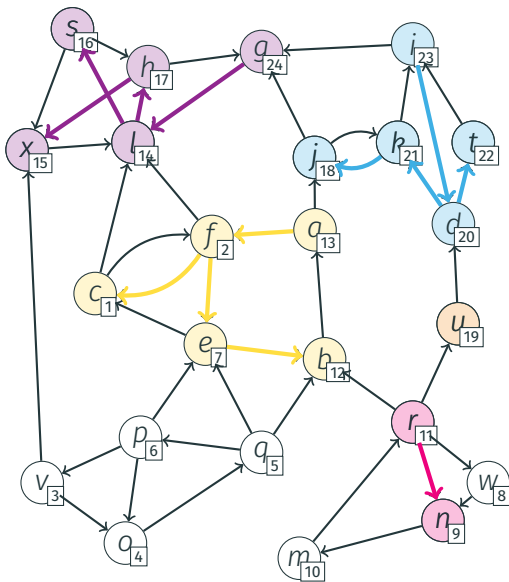


Illustration de la phase 2

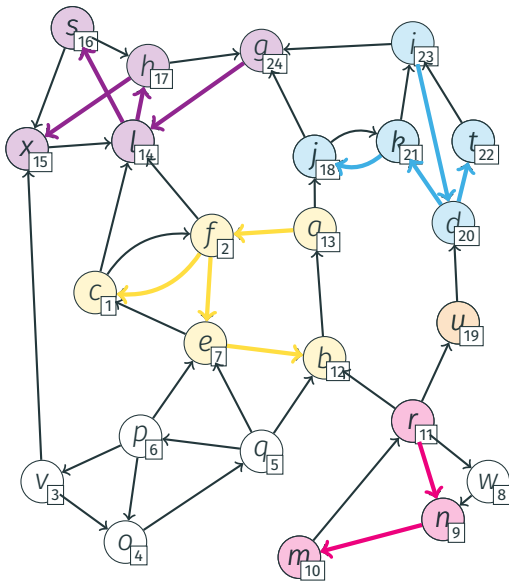


Illustration de la phase 2

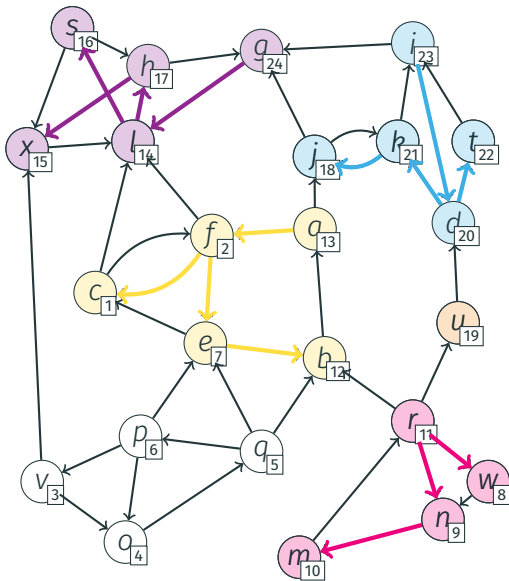


Illustration de la phase 2

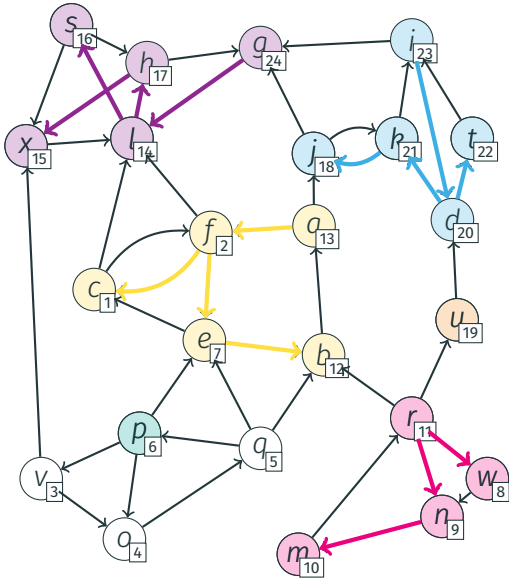


Illustration de la phase 2

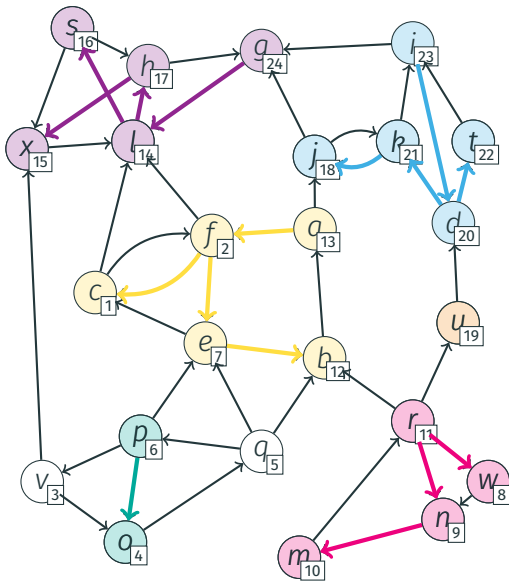


Illustration de la phase 2

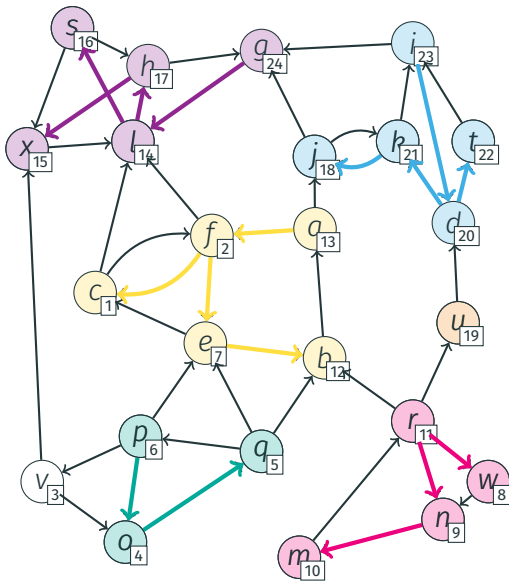
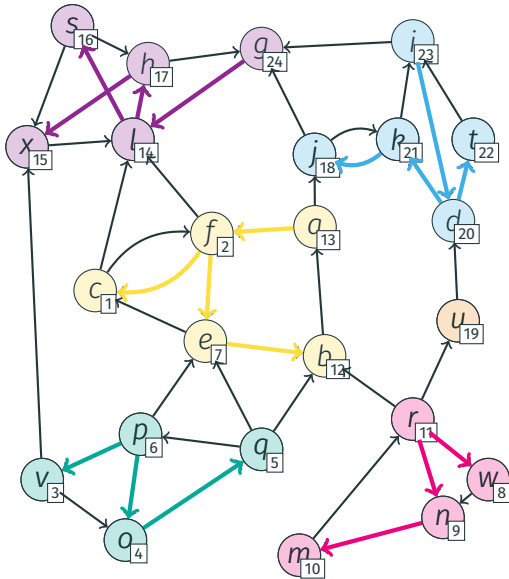


Illustration de la phase 2



Pour toute composante fortement connexe C , on note

$$n^o(C) = \max_{u \in C} n^o(u).$$

Lemme

La numérotation ainsi définie est une numérotation topologique du graphe des composantes fortement connexes.

Théorème

L'algorithme de Kosaraju identifie les composantes fortement connexes.

Preuve de correction (algo. de Kosaraju)

Théorème

L'algorithme de Kosaraju identifie les composantes fortement connexes.

Démonstration.

Raisonnement par récurrence sur le nombre de DFS de la phase 2.

Preuve de correction (algo. de Kosaraju)

Théorème

L'algorithme de Kosaraju identifie les composantes fortement connexes.

Démonstration.

Raisonnement par récurrence sur le nombre de DFS de la phase 2.

Soit u_0 le sommet de plus grand numéro. Montrons que $DFS_{G^T}(u_0)$ renvoie la composante fortement connexe $\overline{u_0}$ de u_0 .

Preuve de correction (algo. de Kosaraju)

Théorème

L'algorithme de Kosaraju identifie les composantes fortement connexes.

Démonstration.

Raisonnement par récurrence sur le nombre de DFS de la phase 2.

Soit u_0 le sommet de plus grand numéro. Montrons que $DFS_{G^T}(u_0)$ renvoie la composante fortement connexe $\overline{u_0}$ de u_0 .

Si $v \in \overline{u_0}$, alors $v \rightsquigarrow_G u_0$. Donc $u_0 \rightsquigarrow_{G^T} v$ et v apparaît dans $DFS_{G^T}(u_0)$.

Preuve de correction (algo. de Kosaraju)

Théorème

L'algorithme de Kosaraju identifie les composantes fortement connexes.

Démonstration.

Raisonnement par récurrence sur le nombre de DFS de la phase 2.

Soit u_0 le sommet de plus grand numéro. Montrons que $DFS_{G^T}(u_0)$ renvoie la composante fortement connexe $\overline{u_0}$ de u_0 .

Si $v \in \overline{u_0}$, alors $v \rightsquigarrow_G u_0$. Donc $u_0 \rightsquigarrow_{G^T} v$ et v apparaît dans $DFS_{G^T}(u_0)$.

Si $v \notin \overline{u_0}$ et v apparaît dans $DFS_{G^T}(u_0)$. Alors, tout $\bar{v}$ apparaît dans $DFS_{G^T}(u_0)$, y compris le sommet v_0 de plus haut n°. Mais alors on aurait $v_0 \rightsquigarrow_G u_0$, ce qui contredirait le caractère topologique des n°s du quotient.

Preuve de correction (algo. de Kosaraju)

Théorème

L'algorithme de Kosaraju identifie les composantes fortement connexes.

Démonstration.

Raisonnement par récurrence sur le nombre de DFS de la phase 2.

Soit u_0 le sommet de plus grand numéro. Montrons que $DFS_{G^T}(u_0)$ renvoie la composante fortement connexe $\overline{u_0}$ de u_0 .

Si $v \in \overline{u_0}$, alors $v \rightsquigarrow_G u_0$. Donc $u_0 \rightsquigarrow_{G^T} v$ et v apparaît dans $DFS_{G^T}(u_0)$.

Si $v \notin \overline{u_0}$ et v apparaît dans $DFS_{G^T}(u_0)$. Alors, tout $\bar{v}$ apparaît dans $DFS_{G^T}(u_0)$, y compris le sommet v_0 de plus haut n°. Mais alors on aurait $v_0 \rightsquigarrow_G u_0$, ce qui contredirait le caractère topologique des n°s du quotient.

Après suppression de $\overline{u_0}$, on applique l'hypothèse de récurrence. □

Parcours en largeur

Le **parcours en largeur** (ou BFS, comme *Breadth First Search*) du graphe peut se décrire comme suit :

Algorithme 6 : Parcours en largeur

Entrées : Graphe $G = (V, E)$, sommet s

Résultat : Visite des sommets accessibles depuis s

```
1  $T \leftarrow \{s\}$  (file servant de collection d'attente)
2 Marquer  $s$ 
3 tant que  $T$  non vide faire
4   Extraire  $v$  de  $T$ 
5   Visiter  $v$ 
6   pour  $w$  voisin de  $v$  non marqué faire
7      $\text{père}(w) \leftarrow v$ 
8      $T \leftarrow T \cup \{w\}$ 
9     Marquer  $w$ 
```



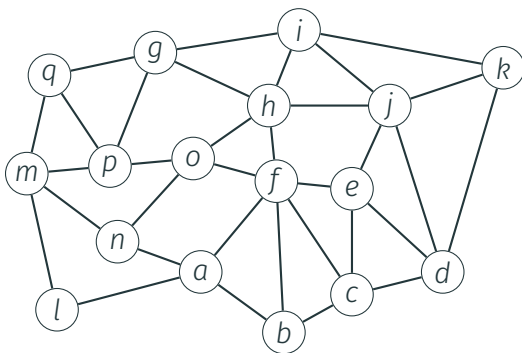
Implémentation du BFS en python

```
class Graph():  
    # [...] Reprendre le code de la classe existante !  
    def bfs(self, r):  
        parent = dict({r:-1})  
        waiting = collections.deque([r])  
        while waiting:  
            v = waiting.popleft()  
            for u in self.neighbors[v]:  
                if u not in parent:  
                    parent[u] = v  
                    waiting.append(u)  
        return parent
```

Parcours en largeur : illustration

Exemple

Parcours en largeur au départ du sommet n .

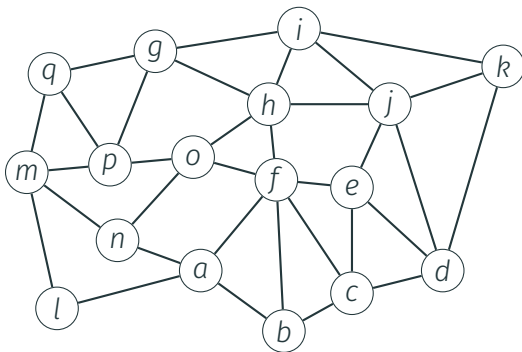


Parcours en largeur : illustration

Exemple

Parcours en largeur au départ du sommet n .

État de la file : $[n]$

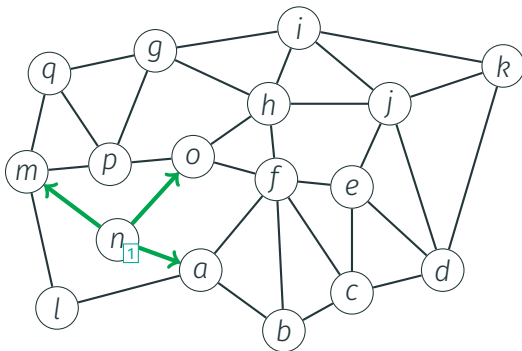


Parcours en largeur : illustration

Exemple

Parcours en largeur au départ du sommet n .

État de la file : $[a, m, o]$

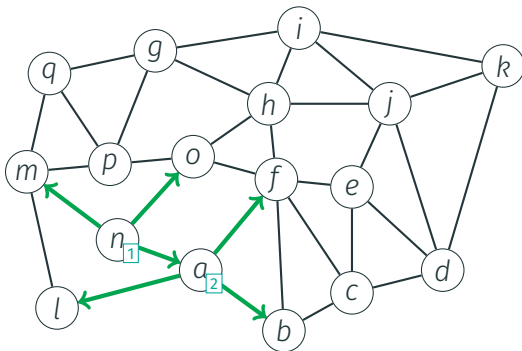


Parcours en largeur : illustration

Exemple

Parcours en largeur au départ du sommet n .

État de la file : $[m, o, b, f, l]$

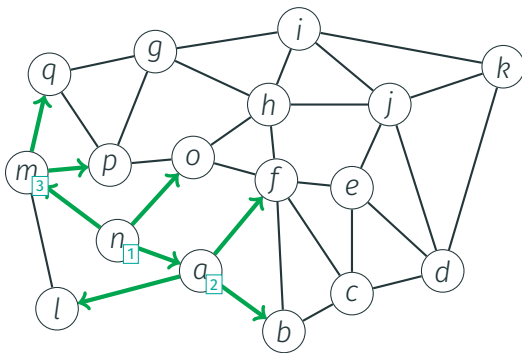


Parcours en largeur : illustration

Exemple

Parcours en largeur au départ du sommet n .

État de la file : $[o, b, f, l, p, q]$

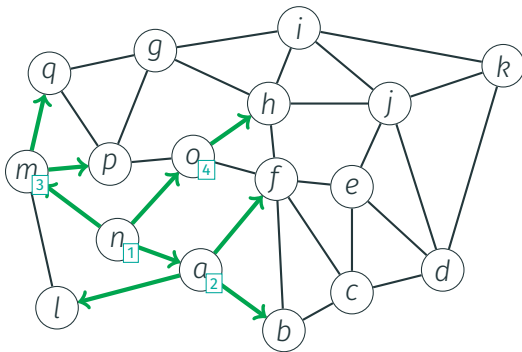


Parcours en largeur : illustration

Exemple

Parcours en largeur au départ du sommet n .

État de la file : $[b, f, l, p, q, h]$

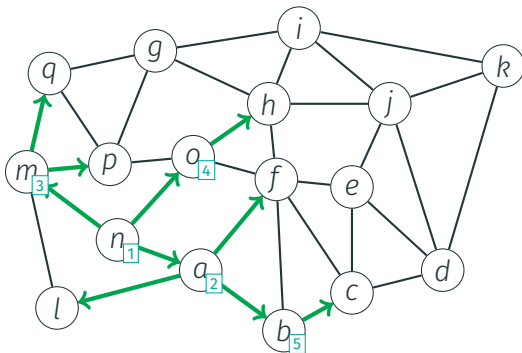


Parcours en largeur : illustration

Exemple

Parcours en largeur au départ du sommet n .

État de la file : $[f, l, p, q, h, c]$

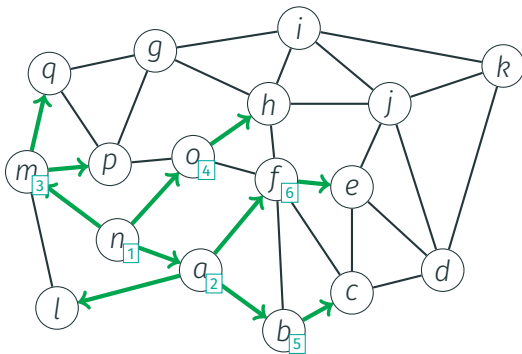


Parcours en largeur : illustration

Exemple

Parcours en largeur au départ du sommet n .

État de la file : $[l, p, q, h, c, e]$

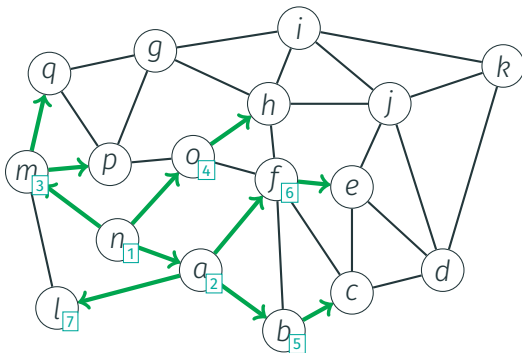


Parcours en largeur : illustration

Exemple

Parcours en largeur au départ du sommet n .

État de la file : $[p, q, h, c, e]$

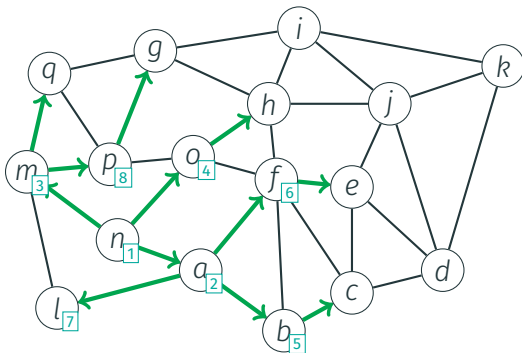


Parcours en largeur : illustration

Exemple

Parcours en largeur au départ du sommet n .

État de la file : $[q, h, c, e, g]$

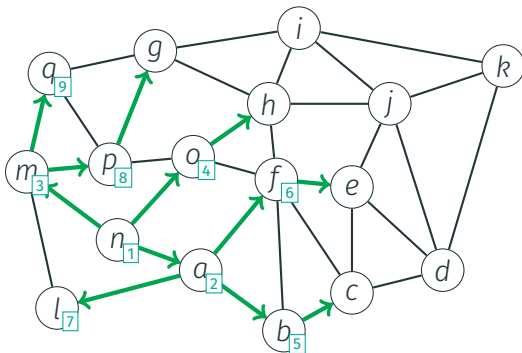


Parcours en largeur : illustration

Exemple

Parcours en largeur au départ du sommet n .

État de la file : $[h, c, e, g]$

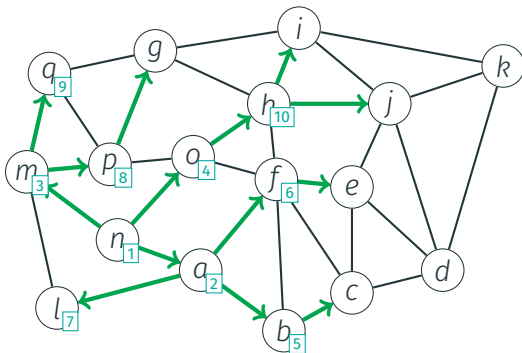


Parcours en largeur : illustration

Exemple

Parcours en largeur au départ du sommet n .

État de la file : $[c,e,g,i,j]$

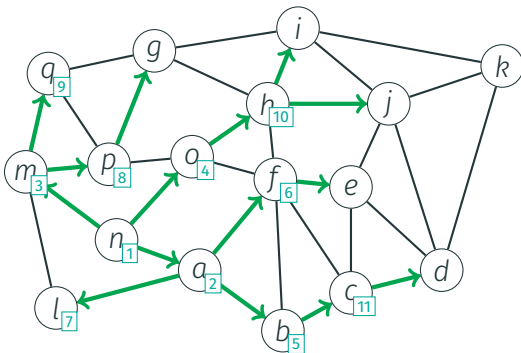


Parcours en largeur : illustration

Exemple

Parcours en largeur au départ du sommet n .

État de la file : $[e,g,i,j,d]$

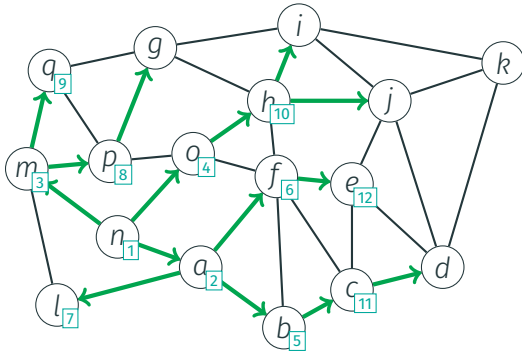


Parcours en largeur : illustration

Exemple

Parcours en largeur au départ du sommet n .

État de la file : $[g,i,j,d]$

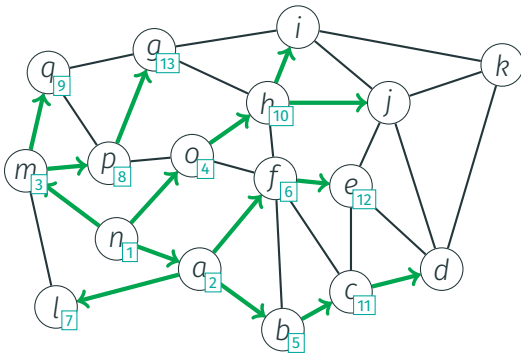


Parcours en largeur : illustration

Exemple

Parcours en largeur au départ du sommet n .

État de la file : $[i,j,d]$

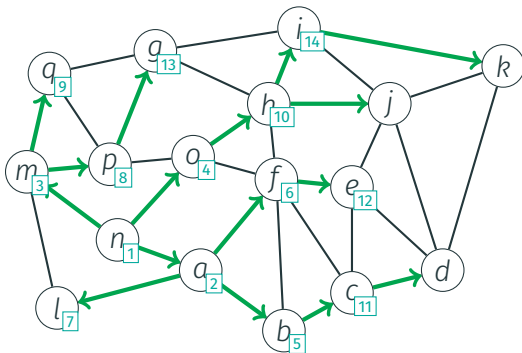


Parcours en largeur : illustration

Exemple

Parcours en largeur au départ du sommet n .

État de la file : $[j,d,k]$

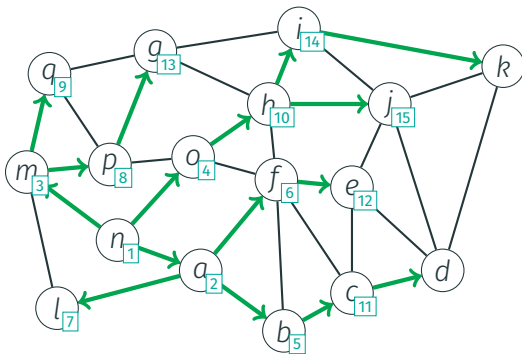


Parcours en largeur : illustration

Exemple

Parcours en largeur au départ du sommet n .

État de la file : $[d,k]$

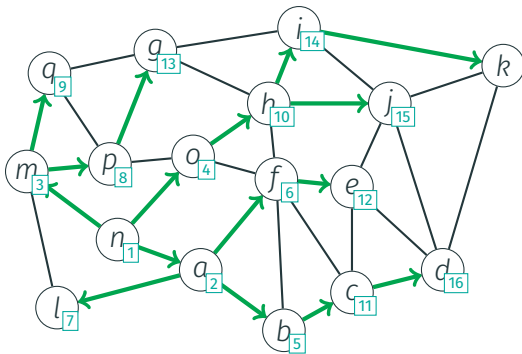


Parcours en largeur : illustration

Exemple

Parcours en largeur au départ du sommet n .

État de la file : $[k]$

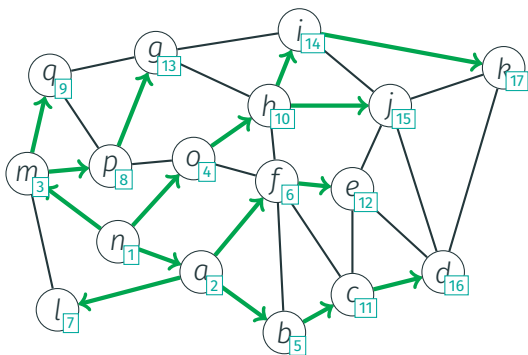


Parcours en largeur : illustration

Exemple

Parcours en largeur au départ du sommet n .

État de la file : $\emptyset$



Application des BFS

Calcul des **plus courts chemins** dans un graphe

Algorithme 7 : Distance dans un graphe unitaire

Entrées : Graphe $G = (V, E)$, sommet s

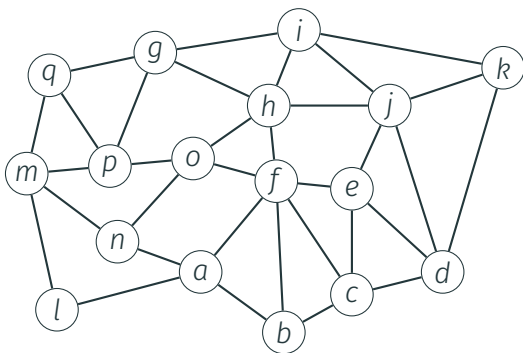
Résultat : Nombre d'arêtes entre s et tout sommet

```
1  $T \leftarrow \{s\}$  ( $T$  est une file)
2 Marquer  $s$ 
3  $d(s) \leftarrow 0$ 
4 tant que  $T$  non vide faire
5     Extraire  $v$  de  $T$ 
6     pour  $w$  voisin de  $v$  non marqué faire
7         père( $w$ )  $\leftarrow v$ 
8          $d(w) \leftarrow d(v) + 1$ 
9          $T \leftarrow T \cup \{w\}$ 
10    Marquer  $w$ 
```

Plus courts chemins : illustration

Exemple

Plus courts chemins au départ du sommet n .

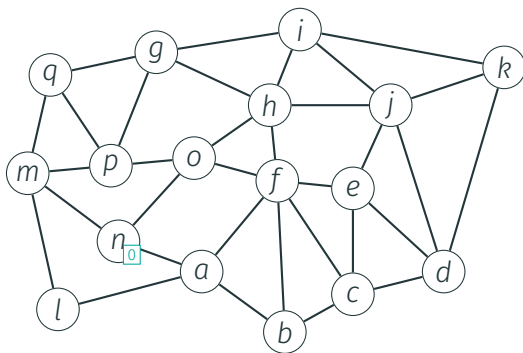


Plus courts chemins : illustration

Exemple

Plus courts chemins au départ du sommet n .

État de la file : $[n]$

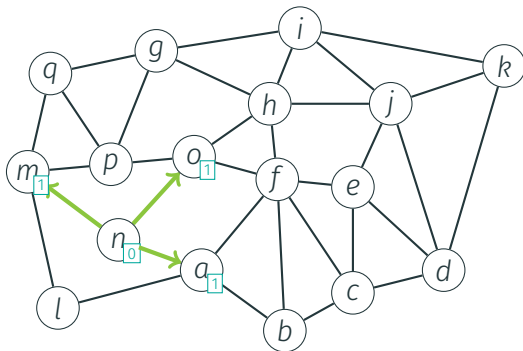


Plus courts chemins : illustration

Exemple

Plus courts chemins au départ du sommet n .

État de la file : $[a, m, o]$

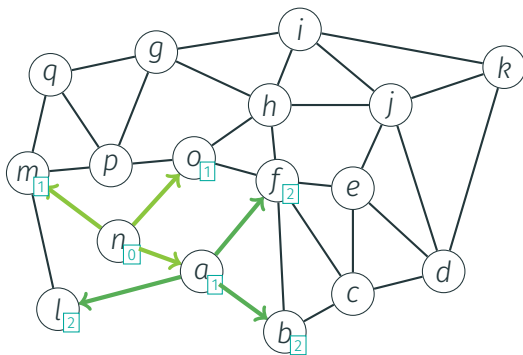


Plus courts chemins : illustration

Exemple

Plus courts chemins au départ du sommet n .

État de la file : $[m,o,b,f,l]$

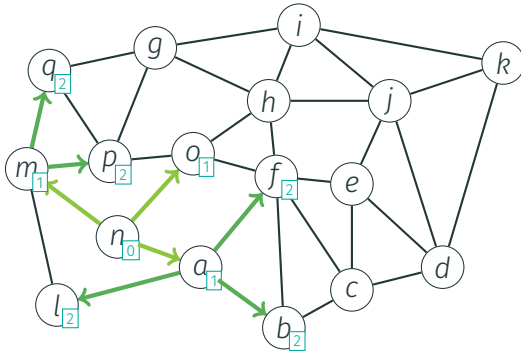


Plus courts chemins : illustration

Exemple

Plus courts chemins au départ du sommet n .

État de la file : $[o, b, f, l, p, q]$

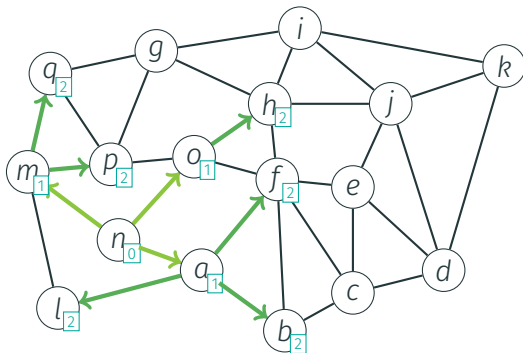


Plus courts chemins : illustration

Exemple

Plus courts chemins au départ du sommet n .

État de la file : $[b,f,l,p,q,h]$

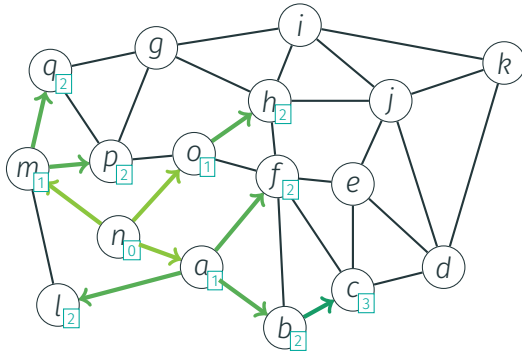


Plus courts chemins : illustration

Example

Plus courts chemins au départ du sommet n .

État de la file : [f,l,p,q,h,c]

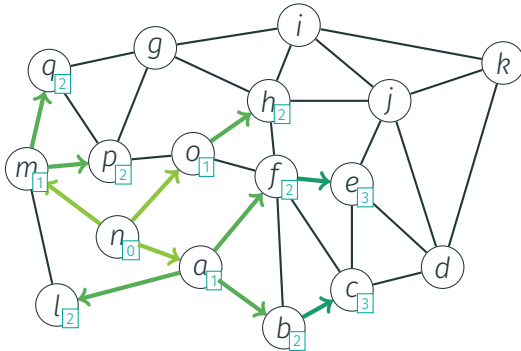


Plus courts chemins : illustration

Exemple

Plus courts chemins au départ du sommet n .

État de la file : $[l, p, q, h, c, e]$

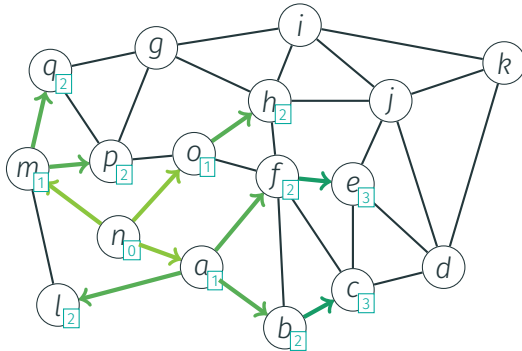


Plus courts chemins : illustration

Exemple

Plus courts chemins au départ du sommet n .

État de la file : $[p,q,h,c,e]$

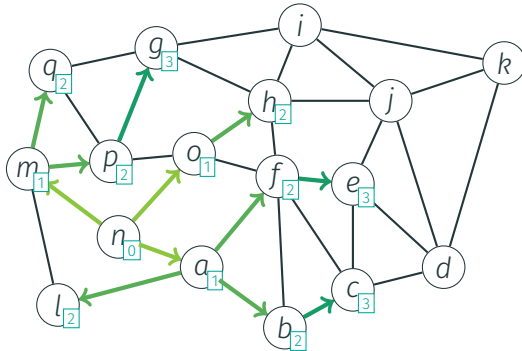


Plus courts chemins : illustration

Exemple

Plus courts chemins au départ du sommet n .

État de la file : $[q,h,c,e,g]$

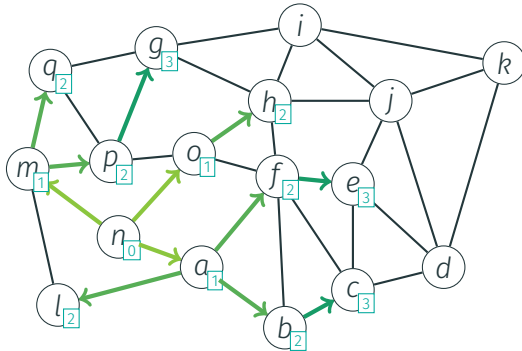


Plus courts chemins : illustration

Exemple

Plus courts chemins au départ du sommet n .

État de la file : $[h,c,e,g]$

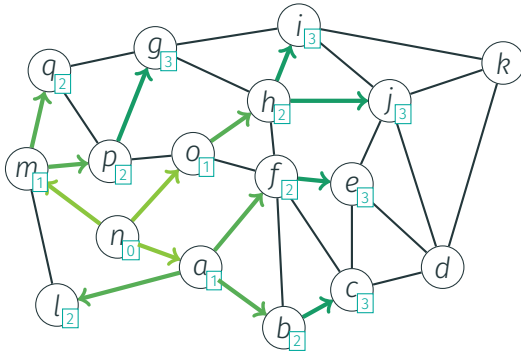


Plus courts chemins : illustration

Exemple

Plus courts chemins au départ du sommet n .

État de la file : $[c,e,g,i,j]$

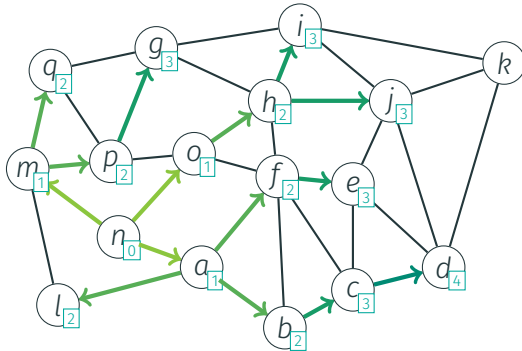


Plus courts chemins : illustration

Example

Plus courts chemins au départ du sommet n .

État de la file : [e,g,i,j,d]

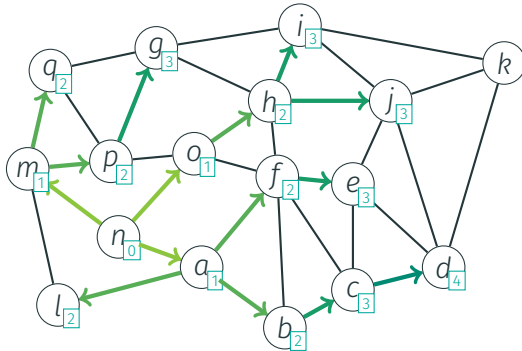


Plus courts chemins : illustration

Example

Plus courts chemins au départ du sommet n .

État de la file : [g,i,j,d]

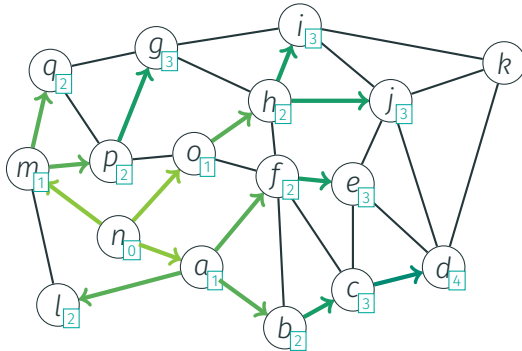


Plus courts chemins : illustration

Example

Plus courts chemins au départ du sommet n .

État de la file : $[i,j,d]$

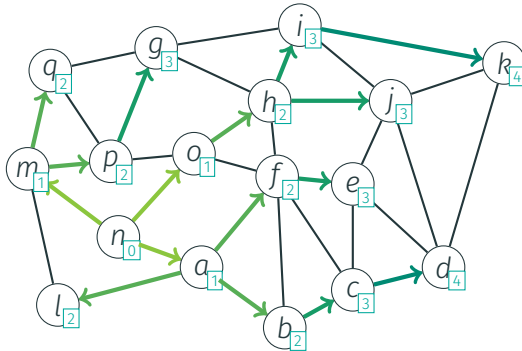


Plus courts chemins : illustration

Example

Plus courts chemins au départ du sommet n .

État de la file : [j,d,k]

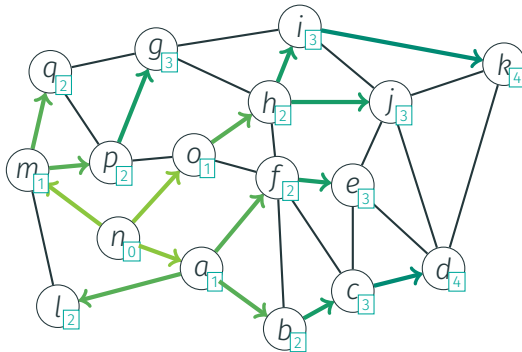


Plus courts chemins : illustration

Exemple

Plus courts chemins au départ du sommet n .

État de la file : $[d,k]$

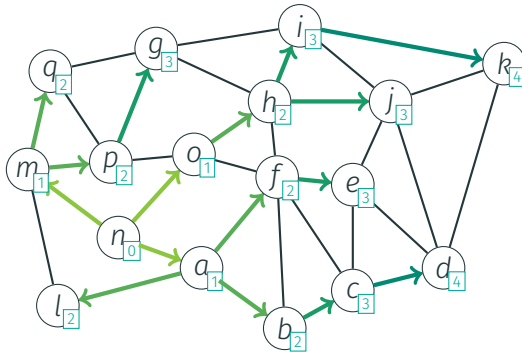


Plus courts chemins : illustration

Exemple

Plus courts chemins au départ du sommet n .

État de la file : $[k]$

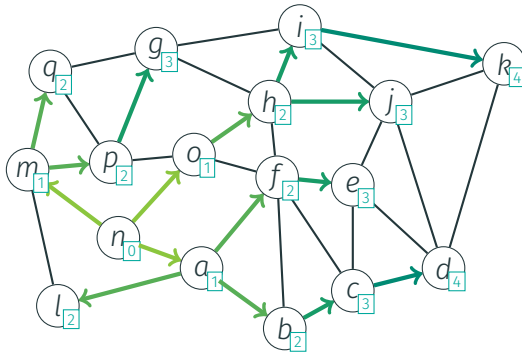


Plus courts chemins : illustration

Example

Plus courts chemins au départ du sommet n .

État de la file : $\emptyset$



Plus courts chemins : illustration

Exemple

Plus courts chemins au départ du sommet n .

